

ICPC Algorithm Field Guide

176 algorithms / structures / standard problem techniques from the photographed UFMG reference index

Written as a recognition-and-recall companion: what it is, the main invariant/recipe, complexity, and two or three quick examples.

Contents

1 Dynamic Programming	3	2.36 Treap	7	3.40 Topological Sort	12
1.1 Divide and Conquer DP	3	2.37 Implicit Treap	7	3.41 Unrooted Euler Tour Tree	12
1.2 Longest Common Subsequence (LCS)	3	2.38 Persistent Implicit Treap	7	3.42 Exact Minimum Vertex Cover (General Graph)	12
1.3 Knapsack (Mochila)	3	2.39 Wavelet Tree	8	3.43 Virtual Tree	12
1.4 SOS DP (Sum Over Subsets)	3				
1.5 Subset Sum	3				
2 Data Structures	3	3 Graphs and Trees	8	4 Mathematics	13
2.1 BIT / Fenwick Tree	3	3.1 Directed MST / AGM Direccionada (Chu-Liu/Edmonds)	8	4.1 2-SAT	13
2.2 BIT 2D	3	3.2 Articulation Points	8	4.2 Interpolation Evaluation at Consecutive Points	13
2.3 BIT with Range Update	3	3.3 Bellman-Ford	8	4.3 Berlekamp-Massey	13
2.4 BIT-Sort Tree	3	3.4 Block-Cut Tree	8	4.4 Binomial Distribution	13
2.5 Dynamic Convex Hull Trick	4	3.5 Blossom (General Graph Matching)	8	4.5 GCD / LCM Convolution	13
2.6 Static / Monotone Convex Hull Trick	4	3.6 Tree Center / Diameter	8	4.6 Coprime Basis	13
2.7 DSU / Union-Find	4	3.7 Centroid of a Tree	8	4.7 Sieve of Eratosthenes / Linear Sieve	13
2.8 Li Chao Tree	4	3.8 Centroid Decomposition	8	4.8 Cycle Detection – Tortoise and Hare	13
2.9 Li Chao Tree – Lazy	4	3.9 Centroid Tree with Precomputed Distances	9	4.9 Division Trick / Quotient Grouping	14
2.10 MergeSort Tree	4	3.10 Dijkstra	9	4.10 Linear Diophantine Equation	14
2.11 Min Queue – Deque	4	3.11 Dinic / Dinitz Max Flow	9	4.11 Extended Euclidean Algorithm	14
2.12 Min Queue – Two Stacks	4	3.12 Dominator Tree	9	4.12 Fast Exponentiation	14
2.13 Order Statistic Set	5	3.13 Euler Path / Euler Cycle	9	4.13 Fast Walsh-Hadamard Transform (FWHT)	14
2.14 Priority Queue DS	5	3.14 Floyd-Warshall	9	4.14 FFT	14
2.15 Range Color	5	3.15 Functional Graph	9	4.15 Gaussian Elimination	14
2.16 RMQ $\langle O(n), O(1) \rangle$ – Min Queue / Cartesian-LCA style	5	3.16 HLD – Edge Values	9	4.16 Gaussian Elimination over $\mathbf{F}_2$ (Gauss-Z2)	14
2.17 SegTreap	5	3.17 HLD – Vertex Values	9	4.17 Numerical Integration	14
2.18 Segment Tree	5	3.18 HLD without Updates	10	4.18 Modular Inverse	15
2.19 Iterative 2D Segment Tree	5	3.19 Hopcroft-Karp	10	4.19 Karatsuba Multiplication	15
2.20 Segment Tree Beats	5	3.20 Tree Isomorphism	10	4.20 Discrete Logarithm	15
2.21 Colored Segment Tree	5	3.21 Johnson's Algorithm	10	4.21 Miller-Rabin Primality Test	15
2.22 Sparse Segment Tree – Lazy	6	3.22 Kosaraju SCC	10	4.22 Multipoint Evaluation and Interpolation	15
2.23 Sparse Segment Tree – $O(q)$ Memory	6	3.23 Kruskal	10	4.23 NTT (Number Theoretic Transform)	15
2.24 Iterative Segment Tree	6	3.24 Kuhn Bipartite Matching	10	4.24 Polynomial and Formal Power Series Operations	15
2.25 Iterative Segment Tree with Lazy Propagation	6	3.25 LCA with Binary Lifting	10	4.25 Pollard's Rho Factorization	15
2.26 Segment Tree PA (Arithmetic Progressions)	6	3.26 LCA with HLD	10	4.26 Product of Two long long Modulo m (Mulmod)	15
2.27 Persistent Segment Tree	6	3.27 LCA with RMQ	11	4.27 Simplex	16
2.28 Persistent Segment Tree with Lazy	6	3.28 Line Tree	11	4.28 Chinese Remainder Theorem (CRT)	16
2.29 Slope Trick	6	3.29 Link-Cut Tree	11	4.29 Euler's Totient φ	16
2.30 Sparse Table	7	3.30 Link-Cut Tree – Edge Values	11		
2.31 Disjoint Sparse Table	7	3.31 Link-Cut Tree – Vertex Values	11	5 Primitives and Reusable Building Blocks	16
2.32 Splay Tree	7	3.32 Max Flow with Lower Bounds	11	5.1 Modular Arithmetic	16
2.33 Implicit Splay Tree	7	3.33 Min-Cost Max-Flow	11	5.2 Big Integer	16
2.34 Split-Merge Set	7	3.34 Prüfer Code	11	5.3 Calendar / Date Arithmetic	16
2.35 SQRT Tree	7	3.35 Rooted Euler Tour Tree	12	5.4 Exact Fraction / Rational	16
		3.36 Sack / DSU on Tree	12	5.5 2D Geometry Primitives	16
		3.37 Stable Marriage (Gale-Shapley)	12	5.6 Integer Geometry Primitives	17
		3.38 Tarjan SCC	12	5.7 3D Geometry Primitives	17
		3.39 Top Tree	12	5.8 Matrix Primitive / Matrix Exponentiation	17
				5.9 Matroid / Matroid Intersection	17

1 Dynamic Programming

1.1 Divide and Conquer DP

Recognize. A DP layer has the form $dp[k][i] = \min_{j < i} (dp[k-1][j] + C(j, i))$ (or max), and the optimal split point is known/provable to be monotone in i .

Idea. Compute one DP row recursively on an interval $[L, R]$. At midpoint M , scan only the candidate split range allowed by the monotonicity bound, store the best split, then recurse left/right with correspondingly narrowed candidate ranges. The optimization is about reducing transitions, not changing the recurrence.

Cost / notes. Typically reduces $O(KN^2)$ to $O(KN \log N)$, and in common monotone settings effectively $O(KN)$ transition scans per level structure; exact bound depends on the cost evaluation. You need a proof of monotone optima (often quadrangle/Monge conditions).

Typical problem statements.

- Partition an array into K groups minimizing a convex/Monge segment cost.
- Split sorted points into K clusters minimizing within-cluster cost.
- Batch jobs into K consecutive batches where each batch has cost $C(l, r)$.

1.2 Longest Common Subsequence (LCS)

Recognize. You need the longest sequence obtainable from two sequences by deleting elements without reordering them.

Idea. Classic DP: $dp[i][j]$ is the LCS length of prefixes. If $a_i = b_j$, extend $dp[i-1][j-1]$; otherwise take $\max(dp[i-1][j], dp[i][j-1])$. To reconstruct, keep parents or recompute choices backwards. If only the length is needed, two rows suffice.

Cost / notes. $O(nm)$ time, $O(nm)$ memory with reconstruction or $O(\min(n, m))$ for length only. For permutations/sparse matches, reductions to LIS can be much faster.

Typical problem statements.

- Find the minimum deletions needed to make two strings equal.
- Compute the longest common subsequence of two DNA strings.
- Given two permutations, find their longest order-consistent common subsequence.

1.3 Knapsack (Mochila)

Recognize. Items have weights/costs and values, and you choose a feasible subset under a capacity or target constraint.

Idea. For 0/1 knapsack, let $dp[w]$ be the best value using total weight at most/exactly w and iterate weights downward for each item so it is used once. Iterate upward for unbounded knapsack. Equivalent formulations minimize weight for a value, count ways, or track reachability.

Cost / notes. $O(nW)$ time and $O(W)$ memory for capacity W . The loop direction is the critical correctness detail; pseudo-polynomial means this is useful only when the numeric bound is moderate.

Typical problem statements.

- Maximize profit with total item weight at most W .
- Can a subset of coins reach exactly sum S ?
- Minimum cost to achieve at least a required total value.

1.4 SOS DP (Sum Over Subsets)

Recognize. States are bitmasks and you need, for every mask, an aggregate over all its submasks or supermasks.

Idea. Start with $f[mask]$ and process bits one by one. For submask sums, whenever bit b is set in $mask$, add the value from $mask \setminus \{b\}$. This is the subset-zeta transform; reverse subtraction is Möbius inversion. Change the transition direction for supermasks.

Cost / notes. $O(2^B)$ time and $O(2^B)$ memory for B bits, instead of enumerating all submasks for every mask (3^B total). Works for sums and many associative/invertible transforms.

Typical problem statements.

- For every feature mask, count objects whose feature set is a subset of it.
- For every mask, count pairs whose bitwise OR/AND satisfies a condition.
- Compute subset convolution-style preprocessing for inclusion–exclusion.

1.5 Subset Sum

Recognize. You only care which sums can be formed by choosing each number at most once.

Idea. Boolean DP is the knapsack specialization $reachable[s] \leftarrow reachable[s] \vee reachable[s - a]$. A very fast implementation stores states in a bitset and performs $bits \mid= bits \ll a$. Meet-in-the-middle is preferable when n is small but values are huge.

Cost / notes. $O(nS)$ scalar DP or roughly $O(nS/w)$ with word-packed bitsets, memory $O(S)$. Meet-in-the-middle gives $O(2^{n/2})$ time/memory tradeoffs.

Typical problem statements.

- Determine whether some subset sums to exactly S .
- List/count all reachable money sums.
- Split numbers into two groups with minimum difference between sums.

2 Data Structures

2.1 BIT / Fenwick Tree

Recognize. You need dynamic prefix sums (or another invertible prefix aggregate) with point updates.

Idea. Store partial sums over ranges determined by the least significant set bit. Updating index i climbs by $i += i \& -i$; a prefix query descends by $i -= i \& -i$. Range sum is $\text{prefix}(r) - \text{prefix}(l-1)$. With coordinate compression it also becomes a frequency/order-statistics tool.

Cost / notes. $O(\log n)$ per update/query, $O(n)$ memory; linear-time construction is possible. Simpler and usually faster than a segment tree when its operation set is enough.

Typical problem statements.

- Point updates and range-sum queries.
- Count inversions while scanning a permutation.
- Maintain frequencies and find the k -th present value by Fenwick binary lifting.

2.2 BIT 2D

Recognize. You need point updates and axis-aligned rectangle prefix/sum queries on a grid.

Idea. Apply the Fenwick idea independently in both coordinates. Updating (x, y) visits all Fenwick ancestors in x and y ; $\text{prefix}(x, y)$ sums the rectangle $[1, x] \times [1, y]$. Any rectangle is obtained with four inclusion–exclusion prefixes.

Cost / notes. $O(\log^2 n)$ per operation and $O(nm)$ memory for a dense $n \times m$ grid. Sparse/offline variants coordinate-compress the Fenwick nodes to avoid dense memory.

Typical problem statements.

- Toggle points on a board and query how many lie in a rectangle.
- Update cell values and query rectangular sums.
- Offline dynamic dominance counting in two dimensions.

2.3 BIT with Range Update

Recognize. You want range addition and prefix/range-sum queries rather than just point addition.

Idea. Represent the difference array with Fenwick trees. For range-add + point-query, one BIT is enough: add at l and subtract at $r+1$. For range-add + range-sum, maintain two BITs so prefix sum becomes $x \cdot B_1(x) - B_2(x)$.

Cost / notes. $O(\log n)$ per update/query, $O(n)$ memory. Carefully derive 0/1-index conventions; most bugs are off-by-one errors in the two-BIT formula.

Typical problem statements.

- Add v to every a_i in $[l, r]$ and ask interval sums.
- Apply many interval increments then query individual values.
- Maintain salaries with batch raises over index ranges.

2.4 BIT-Sort Tree

Recognize. The array is static and queries ask how many values in an index interval are $< k, \leq k$, etc.

Idea. Each Fenwick node stores the sorted list of values contributed to that node. A prefix query walks $O(\log n)$ Fenwick nodes and binary-searches inside each; subtract two prefixes for $[l, r]$. It is conceptually a merge-sort tree organized as a BIT.

Cost / notes. As in the UFMG implementation: $O(n \log^2 n)$ build, $O(\log^2 n)$ query, $O(n \log n)$ memory. Static unless the inner vectors are replaced by dynamic ordered structures.

Typical problem statements.

- Count elements $< x$ in subarray $[l, r]$.
- Binary-search the k -th smallest value in a range using count queries.
- Count points in a 2D rectangle after sorting by one coordinate.

2.5 Dynamic Convex Hull Trick

Recognize. You repeatedly add arbitrary lines $y = mx + b$ and query the minimum/maximum at arbitrary x , without monotone slopes or query order.

Idea. Maintain the lower/upper envelope in an ordered structure (commonly a multiset “LineContainer”). Each line stores where it stops being optimal; insertion removes now-useless neighbors. Query by binary-searching the breakpoint containing x .

Cost / notes. Usually $O(\log n)$ amortized per insertion/query with integer floor-division handled carefully. For fully online arbitrary lines, this is often preferable to the monotone deque CHT; Li Chao is another robust option.

Typical problem statements.

- DP of the form $dp[i] = \min_j (m_j x_i + b_j)$ with unsorted slopes and x_i .
- Maintain offers modeled as linear functions and answer cheapest price at demand x .
- Dynamic maximum dot-product queries after an algebraic line transformation.

2.6 Static / Monotone Convex Hull Trick

Recognize. DP transitions become min/max of lines and line slopes and/or query x values arrive monotonically.

Idea. Keep candidate lines in slope order. When inserting a line, pop the previous line while its intersection makes it permanently useless. If query x is monotone, also pop from the front while the next line is better; otherwise binary-search the hull.

Cost / notes. $O(1)$ amortized insertion and monotone query, hence $O(n)$ total after sorting; or $O(\log n)$ arbitrary queries on a static hull. Avoid floating-point intersections by cross multiplication.

Typical problem statements.

- Optimize $dp[i] = \min_{j < i} (dp[j] + a_j b_i)$ when a_j and b_i are monotone.
- Minimum cost partition with affine transition costs.
- Query the best of many fixed linear tariffs at increasing usage levels.

2.7 DSU / Union-Find

Recognize. You need to maintain connected components under edge additions/merges, with no deletions.

Idea. Each set has a representative parent. **find** uses path compression; **union** attaches the smaller-rank/size tree under the larger. Store extra component metadata at roots when needed (size, sum, min, parity, potential). Rollback DSU deliberately omits path compression.

Cost / notes. Nearly $O(1)$ amortized: $O(\alpha(n))$ per operation with both optimizations, $O(n)$ memory. Rollback DSU is $O(\log n)$ -ish tree depth or $O(\alpha)$ -like with union-by-size and no compression, with $O(1)$ rollback per merge record.

Typical problem statements.

- Process edges and answer whether two vertices are connected.
- Kruskal minimum spanning tree.
- Offline dynamic connectivity using a segment tree over time plus rollback DSU.

2.8 Li Chao Tree

Recognize. You need min/max of many lines at queried x values, especially when slope order is arbitrary but the x domain is known.

Idea. Each segment-tree node stores one line. On insertion, compare the new and stored line at the interval midpoint; keep the better-at-mid line there and recursively send the other line only to the side where it may still win. Query follows one root-to-leaf path and takes the best seen value.

Cost / notes. $O(\log X)$ per line insertion/query for an integer domain of size/range X , $O(\text{number of created nodes})$ memory. Segment-restricted lines can also be inserted on subintervals.

Typical problem statements.

- Online DP transitions $\min_j (m_j x + b_j)$ with arbitrary lines.
- Add cost functions valid for all x and query the cheapest at selected coordinates.
- Maintain maximum of affine functions over a large coordinate range.

2.9 Li Chao Tree – Lazy

Recognize. Many lines become active only on intervals, or you need an offline time/coordinate decomposition around Li Chao operations.

Idea. Combine Li Chao line envelopes with lazy/sparse segment activation: descend only into nodes whose coordinate interval is affected, and keep the usual one-line-per-node dominance invariant. The exact notebook variant is useful when line information is propagated or attached to intervals rather than globally inserted.

Cost / notes. Typically $O(\log^2 X)$ for segment-line insertion and $O(\log X)$ query; global insertion remains $O(\log X)$. Sparse allocation avoids paying for the full domain.

Typical problem statements.

- A linear offer is valid only for $x \in [l, r]$; query the cheapest active offer.
- Offline DP where each transition line applies only to a future index interval.
- Dynamic piecewise-linear lower-envelope queries.

2.10 MergeSort Tree

Recognize. The array is mostly/static and interval queries need order information, not just a monoid such as sum/min.

Idea. Build a segment tree where every node stores its covered values sorted. Decompose $[l, r]$ into $O(\log n)$ nodes and binary-search each list to count values below/above a threshold. Fractional cascading or wavelet trees can improve variants.

Cost / notes. $O(n \log n)$ build/memory and $O(\log^2 n)$ threshold-count query. A k -th statistic can be found by binary-searching the value domain, often $O(\log V \log^2 n)$.

Typical problem statements.

- Count values $\leq x$ inside $[l, r]$.
- Find the median/ k -th smallest in a subarray.
- Count points in an orthogonal rectangle after sorting by one axis.

2.11 Min Queue – Deque

Recognize. You need the minimum/maximum of every sliding window while pushing at one end and expiring from the other.

Idea. Keep a deque of candidate indices with values monotonically increasing for minima. Before pushing i , remove worse values from the back; before answering, remove front indices outside the window. The front is always the current optimum.

Cost / notes. $O(1)$ amortized per push/pop/query and $O(\text{window})$ memory because each index enters and leaves once. Keep indices, not only values, when duplicates/expiration matter.

Typical problem statements.

- Minimum of every length- k subarray.
- Longest subarray where $\max - \min \leq K$ using two monotone deques.
- DP where $dp[i]$ takes a min over the previous k states.

2.12 Min Queue – Two Stacks

Recognize. You want a queue supporting push, pop-front and min/max, and a stack-based implementation is convenient.

Idea. Use two stacks: input and output. Each stack item stores its value and the aggregate minimum of the stack up to that point. Push to input; when output is empty, transfer all input elements to output. The queue minimum is $\min(\text{top-aggregate of both stacks})$.

Cost / notes. $O(1)$ amortized per operation, $O(n)$ memory. The technique generalizes to any associative aggregate that can be maintained on each stack (SWAG).

Typical problem statements.

- Sliding-window minimum with explicit queue operations.

- Maintain gcd/min/max of a FIFO stream.
- Two-pointer search where the current interval must satisfy an aggregate predicate.

2.13 Order Statistic Set

Recognize. You need an ordered set/multiset with “rank of key” and “element by rank” in addition to insert/erase.

Idea. Use a balanced BST augmented with subtree sizes; in GCC, PBDS `tree` provides `order_of_key(x)` and `find_by_order(k)`. For duplicates, store pairs $(value, unique_id)$ or use a Fenwick tree over compressed values.

Cost / notes. $O(\log n)$ insertion, deletion, rank and k -th query. PBDS is non-standard GNU C++; know whether your contest compiler allows it.

Typical problem statements.

- Maintain a dynamic median.
- Count how many active numbers are smaller than x .
- Insert/delete scores and answer the player with rank k .

2.14 Priority Queue DS

Recognize. Updates to some rollback-capable data structure have priorities, and at any time you need the state containing the currently selected/high-priority updates rather than only stack-order undo.

Idea. The notebook technique stores update operations in a priority-queue-like hierarchy and replays/rolls back a bounded number when priority order changes. It requires the underlying update to support rollback and *not* rely on amortized operations, so each local undo/reapply has a predictable cost.

Cost / notes. Complexity depends on the wrapped structure and implementation; the purpose is to obtain logarithmic/polylog overhead around prioritized insertion/removal of reversible updates. Treat it as an advanced offline/dynamic-structure adapter, not a normal binary heap.

Typical problem statements.

- Maintain a rollback DSU state induced by currently highest-priority constraints.
- Dynamically choose which reversible geometric/events updates are active according to priority.

2.15 Range Color

Recognize. An array is partitioned into maximal constant-color intervals; updates repaint a whole range and queries ask a position’s color or which intervals were overwritten.

Idea. Store disjoint intervals $(l, r, color)$ ordered by l (ODT-style). Split intervals at update boundaries, erase every fully covered interval, return/process them, then insert the new colored interval and optionally merge equal neighbors. Each erased interval pays for itself amortized.

Cost / notes. In the notebook version, $O(\log q)$ amortized repaint over q operations and $O(\log q)$ point query, assuming intervals are destroyed/created in the usual amortized manner.

Typical problem statements.

- Repaint seats $[l, r]$ and report which colors were replaced.
- Maintain ownership of positions under interval assignments.
- Count/process maximal segments affected by range overwrite operations.

2.16 RMQ $\langle O(n), O(1) \rangle$ – Min Queue / Cartesian-LCA style

Recognize. The array is static and you need very many range minimum/maximum queries with true constant query time.

Idea. Linear-preprocess RMQ by reducing it to LCA on the Cartesian tree / Euler depth sequence, then exploit the ± 1 structure with block decomposition and precomputed block types (Farach-Colton–Bender style). The notebook implementation packages this as linear build + constant query.

Cost / notes. $O(n)$ preprocessing and memory, $O(1)$ query. Harder to implement than sparse table, so it matters mainly when $O(1)$ queries or linear memory/preprocessing is specifically useful.

Typical problem statements.

- Millions of static range-minimum queries.
- LCA after Euler tour with optimal asymptotics.
- Constant-time minima used inside another theoretically tight algorithm.

2.17 SegTreap

Recognize. You need a sparse 2D segment-tree-like structure: one coordinate is indexed by a segment tree and the other needs dynamic ordered aggregation.

Idea. The UFMG notebook describes it as equivalent to a sparse 2D segment tree. Conceptually, segment-tree decomposition handles one dimension while treaps inside nodes represent only actually present points/keys in the second dimension, allowing dynamic sparse updates and rectangle-like queries without dense n^2 storage.

Cost / notes. Usually polylogarithmic, commonly $O(\log^2 n)$ expected per point update/query and near $O(q \log n)$ created storage for q sparse operations; exact supported aggregate follows the implementation.

Typical problem statements.

- Dynamic 2D point updates with rectangle sums/counts.
- Maintain sparse points and count those in $[x_1, x_2] \times [y_1, y_2]$.

2.18 Segment Tree

Recognize. You need range queries for an associative merge and point updates, or range updates with lazy propagation.

Idea. Each node stores the aggregate of an interval; children partition it. Query decomposes a range into $O(\log n)$ canonical nodes. For range updates, store a lazy tag describing an operation still to be pushed to children, and define how tags compose and how they affect node aggregates.

Cost / notes. $O(n)$ build, $O(\log n)$ point update/range query; lazy range updates are also $O(\log n)$ when the tag can be applied to a whole node in $O(1)$.

Typical problem statements.

- Dynamic range sum/min/max queries.
- Range add + range maximum.
- Find the first index where a prefix aggregate crosses a threshold by descending the tree.

2.19 Iterative 2D Segment Tree

Recognize. You need dynamic rectangle aggregates on a moderate dense grid and can afford roughly $O(nm)$ memory.

Idea. Use an iterative segment tree in both dimensions: an outer tree over rows, with an inner tree/array over columns at each outer node. A rectangle query decomposes both axes into logarithmically many canonical intervals.

Cost / notes. Typically $O(nm)$ memory/build scale and $O(\log n \log m)$ per point update or rectangle query. Constants can be high; 2D BIT is simpler for sums, while compressed/sparse structures are better for sparse coordinates.

Typical problem statements.

- Point-update grid values and query rectangle sums.
- Dynamic 2D min/max over a matrix.

2.20 Segment Tree Beats

Recognize. Range operations such as `chmin`, `chmax`, add and sum cannot be represented by a simple one-value lazy tag, but often affect most elements in a node uniformly.

Idea. For range `chmin`, store maximum, second maximum, count of maximum and sum. If update value x lies between max and second-max, only the current maxima change, so update the node lazily; otherwise recurse. Symmetric metadata handles `chmax`. The “beats” invariant limits how often expensive descents happen.

Cost / notes. Classic range `chmin/chmax/add/sum` variants run in amortized about $O((n + q) \log n)$ or $O(\log^2 n)$ -style bounds depending on operations/proof. Implementation invariants are delicate.

Typical problem statements.

- For each query cap all a_i in $[l, r]$ by x , then ask sums.
- Support range add, range min/max clamp and range sum.

2.21 Colored Segment Tree

Recognize. Every position has both a numeric value and a color; operations target only positions of one color and may recolor them.

Idea. The notebook keeps a sparse segment-tree root per color. Each root stores count, value sum and lazy addition for positions of that color. Query/update acts on one color; repaint moves/merges the relevant segment-tree nodes from color c_1 to c_2 instead of touching positions individually.

Cost / notes. Notebook bounds: $O(n \log n)$ build space/time, $O(\log n)$ query and value update, $O(\log n)$ amortized repaint. Colors are indexed and sparse roots prevent full $colors \times n$ memory.

Typical problem statements.

- Sum values of red elements in $[l, r]$, add x to them, and recolor red to blue.
- Maintain category-specific interval totals while categories change by range.

2.22 Sparse Segment Tree – Lazy

Recognize. The coordinate domain is huge (e.g. $[0, 10^9]$ or $[0, 2^{60}]$), but only a small number of intervals are ever touched, and range updates are needed.

Idea. Represent segment-tree nodes with pointers and create children only when an operation descends into them. Store the same aggregate/lazy tags as a normal tree; an absent child represents the untouched default interval and can be materialized on demand.

Cost / notes. $O(\log U)$ nodes/time per point operation over universe size U ; range updates/queries create only visited nodes, typically $O(q \log U)$ total memory. Be careful that pushing lazy tags can create children.

Typical problem statements.

- Range-add/query over coordinates up to 10^{18} with only 10^5 operations.
- Maintain covered length or counts on a sparse integer line.

2.23 Sparse Segment Tree – $O(q)$ Memory

Recognize. You need segment-tree behavior on a huge universe, but want memory proportional to the number of operations rather than $q \log U$ in favorable specialized cases.

Idea. Use an implicit/dynamic representation that creates only structurally necessary breakpoints/nodes and often compresses chains of untouched intervals. The notebook variant is engineered so a sequence of q operations allocates $O(q)$ nodes while preserving logarithmic navigation over coordinates.

Cost / notes. Target is $O(q)$ memory with logarithmic operation time (implementation-specific). Use it when the ordinary implicit segment tree’s $O(q \log U)$ allocation is too large.

Typical problem statements.

- 10^6 sparse point/range operations over a 64-bit coordinate universe under a tight memory limit.
- Dynamic interval aggregation where only query/update boundaries matter.

2.24 Iterative Segment Tree

Recognize. You need a compact, fast segment tree for point updates and associative range queries.

Idea. Store leaves in $t[1..2n]$ and parents below them. Point update changes one leaf then recomputes ancestors. For query $[l, r]$, move two pointers upward; whenever one pointer is a right/left boundary, include that node and advance it. Preserve left/right accumulator order for non-commutative operations.

Cost / notes. $O(n)$ build, $O(\log n)$ query/update, about $2n$ memory and excellent constants. Range lazy operations require a more elaborate iterative lazy tree.

Typical problem statements.

- Point assignment + range sum/min/max.
- Range matrix product with point updates.
- Find aggregate on many intervals with minimal implementation overhead.

2.25 Iterative Segment Tree with Lazy Propagation

Recognize. You need range updates and range queries but prefer the memory/cache behavior of an iterative tree.

Idea. Keep tree values and lazy tags in arrays. Before touching boundary leaves, push pending tags from the root down along their ancestor paths; after an update, rebuild affected ancestors upward while accounting for pending tags. Half-open intervals simplify the iterative traversal.

Cost / notes. $O(\log n)$ range update/query and $O(n)$ memory, with smaller constants than pointer recursion when implemented carefully. More error-prone than recursive lazy propagation.

Typical problem statements.

- Range add + range sum on large n, q .

- Range assignment/add + min/max with strict performance constraints.

2.26 Segment Tree PA (Arithmetic Progressions)

Recognize. Range updates assign or add an arithmetic progression across $[l, r]$, and queries ask interval sums.

Idea. A lazy tag is itself an AP: first term plus common difference. For a node, its total contribution is computed with the AP sum formula; when pushing to the right child, shift the first term by the number of positions in the left child. The notebook supports both setting and adding an AP, with set overriding/composing add correctly.

Cost / notes. Notebook: $O(n)$ build and $O(\log n)$ for AP set, AP add and range-sum query. Tag composition and the right-child phase shift are the key details.

Typical problem statements.

- Add $1, 2, 3, \dots$ across every queried interval and ask sums.
- Assign a linear ramp $A, A + R, \dots$ on $[l, r]$.
- Apply distance-based linear contributions over intervals.

2.27 Persistent Segment Tree

Recognize. You need to query old versions after updates, or turn prefix versions into range order-statistic queries.

Idea. Path-copying: an update creates new nodes only along the root-to-leaf path and reuses untouched subtrees from the previous root. Keep one root per version. For a frequency tree, subtract counts between prefix roots $root_r - root_{l-1}$ while descending to find a k -th value.

Cost / notes. $O(\log n)$ new nodes/time per point update, $O(n + q \log n)$ memory. Queries are $O(\log n)$. Coordinates are usually compressed.

Typical problem statements.

- Query the k -th smallest value in subarray $[l, r]$.
- Support “time travel”: update an array and query any previous version.
- Count values in a range using prefix-version differences.

2.28 Persistent Segment Tree with Lazy

Recognize. Old versions must survive even though updates affect whole ranges.

Idea. Combine path-copying with lazy propagation: never mutate a node shared by an older version. Clone any node whose aggregate/tag will change, and when pushing a lazy tag, clone affected children before modifying them. Untouched subtrees remain shared.

Cost / notes. Usually $O(\log n)$ to $O(\log^2 n)$ new work/nodes per range update depending on the lazy operation and implementation; range query remains logarithmic. Memory can grow quickly, so know the version/operation bound.

Typical problem statements.

- Range-add an array while retaining every historical version for range-sum queries.
- Branch from an old version, apply interval updates, then compare versions.

2.29 Slope Trick

Recognize. A DP objective is a convex piecewise-linear function on one variable, built by repeatedly adding terms like $|x - a|$, $\max(0, x - a)$, shifting domains, or taking prefix minima.

Idea. Represent the derivative breakpoints rather than all function values. Two heaps (left/right breakpoints) plus an additive constant maintain the convex function under standard operations. Adding an absolute-value term inserts/rebalances breakpoints; “slide/min over $y \leq x$ ” shifts/removes one side.

Cost / notes. Common operations are $O(\log n)$, memory $O(n)$. The hard part is recognizing that the DP state is convex and translating recurrence operations into slope-trick primitives.

Typical problem statements.

- Choose nondecreasing adjusted values minimizing total absolute deviation.
- Scheduling penalty DP with movement $|x - a_i|$.
- Minimize piecewise-linear convex cost under interval/monotonic constraints.

2.30 Sparse Table

Recognize. The array is static and range queries use an idempotent operation such as min, max, gcd, bitwise AND/OR.

Idea. Precompute $st[k][i]$ for length 2^k . For query $[l, r]$, take two overlapping blocks of length $2^{\lfloor \log_2 len \rfloor}$ that cover both ends; idempotence makes overlap harmless.

Cost / notes. $O(n \log n)$ preprocessing/memory and $O(1)$ query. For non-idempotent associative operations such as sum, use a disjoint sparse table or decompose into $O(\log n)$ blocks.

Typical problem statements.

- Static range minimum queries.
- GCD of many subarrays.
- Static range maximum for binary-search predicates.

2.31 Disjoint Sparse Table

Recognize. The array is static, queries need any associative operation, but overlapping-block sparse-table queries are invalid (e.g. sum, product, matrix multiply).

Idea. At each power-of-two level, split into blocks and precompute suffix aggregates on the left half and prefix aggregates on the right half. For query (l, r) , find the highest bit where l and r differ; exactly one suffix and one prefix from that level combine to the answer.

Cost / notes. $O(n \log n)$ preprocessing/memory and $O(1)$ query for any associative operation. Preserve operand order for non-commutative monoids.

Typical problem statements.

- Static range sums in constant query time.
- Range matrix products.
- Static concatenation/hash-like associative queries where overlap is not allowed.

2.32 Splay Tree

Recognize. You need an ordered BST with split/join, predecessor/successor, or access patterns where recently used keys should become cheap.

Idea. After access/insertion, rotations “splay” the touched node to the root using zig/zig-zig/zig-zag steps. This self-adjustment gives amortized logarithmic operations without stored random priorities or balance factors. Split by splaying a boundary; join by splaying the maximum of the left tree then attaching the right.

Cost / notes. $O(\log n)$ amortized per operation, but a single operation can be $O(n)$. Powerful building block for link-cut trees; do not use when strict worst-case per operation is required.

Typical problem statements.

- Ordered set with split/merge and aggregate metadata.
- Implement a link-cut tree’s auxiliary preferred paths.

2.33 Implicit Splay Tree

Recognize. You manipulate a sequence by position: reverse/cut/insert ranges, query range aggregates, etc.

Idea. Keys are implicit: subtree size determines the in-order index. Splay the node at a chosen rank; isolate interval $[l, r]$ by splaying its boundary nodes, then apply lazy reversal/addition/aggregate updates to the isolated subtree.

Cost / notes. $O(\log n)$ amortized per split/join/range operation, $O(n)$ memory. Maintain subtree sizes and push lazy tags before rotations/descent.

Typical problem statements.

- Text editor: cut/paste/reverse substrings.
- Dynamic sequence range sum with insert/delete by index.
- Reverse segments of a permutation online.

2.34 Split-Merge Set

Recognize. You maintain many sets of integer positions/keys and need fast destructive union plus splitting by a key/range, often with aggregate information.

Idea. Use an implicit/sparse segment-tree representation of a set over the key universe. Merge two trees recursively, reusing nodes where only one side exists; split partitions nodes according to a boundary. Because

nodes are physically moved/reused, total work can be bounded by how often structure changes rather than by iterating all elements.

Cost / notes. Typical merge/split costs are proportional to nodes traversed and often amortize to $O(\log U)$ per moved structural event; the notebook is designed for efficient set union/split on a coordinate universe.

Typical problem statements.

- Maintain disjoint groups of positions, merge groups, and split by value threshold.
- DSU-like component sets where components must later be divided by key.

2.35 SQRT Tree

Recognize. You need static range queries for an arbitrary associative operation, very low query latency, and less memory than a full sparse table.

Idea. Recursively divide the array into roughly square-root-sized blocks. Store prefix/suffix aggregates inside blocks and aggregates between blocks; a query combines a small constant number of precomputed pieces and recurses on a much smaller level.

Cost / notes. Notebook: $O(n \log \log n)$ build/memory scale and $O(\log \log n)$ query, practically near constant because $\log \log n$ is tiny. Works for associative, not necessarily idempotent, operations.

Typical problem statements.

- Huge number of static range sums/products with constant-like query time.
- Static range matrix multiplication under tight latency.

2.36 Treap

Recognize. You need an ordered BST with simple split/merge and expected logarithmic balance.

Idea. Each node has a BST key and random heap priority. Split recursively by key; merge recursively by priority assuming all keys on the left are smaller. Insert/delete can be expressed via split/merge. Augment nodes with subtree size/sum/min and lazy tags as needed.

Cost / notes. $O(\log n)$ expected per operation, $O(n)$ memory. Random priorities are the balance guarantee; use a good RNG and update metadata after every structural change.

Typical problem statements.

- Ordered set supporting split at key x and merge back.
- Dynamic interval set with subtree aggregates.

2.37 Implicit Treap

Recognize. You need a mutable sequence with operations addressed by index rather than key.

Idea. Remove explicit keys; in-order position is determined by subtree sizes. Split by the first k elements, merge adjacent sequences, and isolate an interval using two splits. Lazy flags support reversal, add/assign, etc.; node aggregates support range queries.

Cost / notes. $O(\log n)$ expected per split/merge and hence per usual range edit/query, $O(n)$ memory. Often the cleanest ICPC “rope” structure.

Typical problem statements.

- Reverse or rotate arbitrary subarrays.
- Insert/delete elements at arbitrary positions and query range sums.
- Cut a substring and paste it elsewhere.

2.38 Persistent Implicit Treap

Recognize. You need sequence editing by position while preserving previous versions.

Idea. Use implicit-treap split/merge, but path-copy every node that would be changed and share untouched subtrees between versions. Lazy tags must also obey persistence: clone before pushing/modifying shared children. Keep a root for each version/branch.

Cost / notes. $O(\log n)$ expected new nodes per local edit in simple variants; range edits can clone additional paths. Memory is proportional to updates times logarithmic structural changes.

Typical problem statements.

- Versioned text editor with undo/time-travel queries.
- Maintain many historical permutations under split/merge/reverse.

2.39 Wavelet Tree

Recognize. The array is static (or mostly static) and queries ask order statistics/frequencies by both index range and value range.

Idea. Recursively partition values around a midpoint. At each node store a prefix count telling how many elements went left; this maps an original subarray $[l, r]$ to corresponding ranges in the children. Descend by value or by desired rank. Wavelet matrices are an iterative variant.

Cost / notes. $O(n \log \sigma)$ build/memory and $O(\log \sigma)$ per k -th, count- $\leq x$, or value-frequency query, where σ is value-domain size after compression.

Typical problem statements.

- k -th smallest element in subarray $[l, r]$.
- Count values in both index range $[l, r]$ and value range $[a, b]$.
- Frequency of a value in a subarray.

3 Graphs and Trees

3.1 Directed MST / AGM Direcionada (Chu–Liu/Edmonds)

Recognize. You need a minimum-cost directed spanning arborescence rooted at r : every other vertex has exactly one incoming selected edge and is reachable from r .

Idea. Choose the cheapest incoming edge of every non-root vertex. If these edges are acyclic, they form the optimum. If they contain a directed cycle, contract that cycle, adjust incoming edge costs by subtracting the chosen in-edge cost of their destination, solve recursively/on the contracted graph, then expand the cycle.

Cost / notes. Classic implementation is $O(VE)$; faster variants exist. First ensure every vertex can be reached from the root; unlike Kruskal/Prim, direction and the designated root matter.

Typical problem statements.

- Choose one parent dependency for every node so all are reachable from a root with minimum total cost.
- Build the cheapest directed broadcast/arborescence network from a source.

3.2 Articulation Points

Recognize. You need vertices whose deletion increases the number of connected components of an undirected graph.

Idea. DFS with discovery time $tin[v]$ and $low[v]$, the earliest ancestor reachable from v ’s DFS subtree using at most one back edge. For non-root v , a child u with $low[u] \geq tin[v]$ proves v is an articulation point. The DFS root is articulation iff it has at least two DFS children.

Cost / notes. $O(V + E)$. In multigraphs, identify the parent *edge*, not merely parent vertex, so parallel edges are treated correctly. The same low-link framework finds bridges.

Typical problem statements.

- Find routers whose failure disconnects a network.
- Count vertices that separate at least two regions of a map.
- Build biconnected/block-cut decomposition.

3.3 Bellman–Ford

Recognize. Single-source shortest paths may contain negative edge weights, or you need to detect reachable negative cycles.

Idea. Initialize $dist[s] = 0$ and relax every edge $V - 1$ times; each full pass propagates shortest paths using one more edge. A relaxation on pass V means a reachable negative cycle can improve indefinitely. To mark all vertices affected by such cycles, start from vertices relaxed on the extra pass and traverse outgoing edges.

Cost / notes. $O(VE)$ time, $O(V)$ memory. Stop early if a pass makes no changes. For difference constraints, a super-source connects to all variables with zero-cost edges.

Typical problem statements.

- Shortest paths with negative tolls but no negative cycle.
- Decide whether a system of difference constraints is feasible.
- Find vertices whose shortest distance is $-\infty$ due to a reachable negative cycle.

3.4 Block–Cut Tree

Recognize. You need to reason about articulation points and biconnected components as a tree.

Idea. Run low-link DFS and maintain an edge/vertex stack. Whenever $low[child] \geq tin[v]$, pop one biconnected block. Build a bipartite tree whose nodes are original articulation vertices and biconnected components, connecting an articulation to every block containing it. Non-articulation vertices belong to exactly one block.

Cost / notes. $O(V + E)$ construction. The block-cut graph of each connected component is a tree; preprocess it with LCA/HLD to turn “paths avoiding/removing vertices” questions into tree questions.

Typical problem statements.

- How many articulation vertices lie on every path from u to v ?
- Answer connectivity queries after deleting one vertex.
- Count biconnected components separating pairs.

3.5 Blossom (General Graph Matching)

Recognize. You need maximum-cardinality matching in a general undirected graph, so the graph is not bipartite and odd cycles break ordinary augmenting-path BFS.

Idea. Edmonds’ blossom algorithm searches for augmenting paths while contracting odd alternating cycles (blossoms) into supervertices. When an augmenting path is found, expand contractions and flip matched/unmatched edges along the path. LCA-like bookkeeping identifies the base of a blossom during BFS.

Cost / notes. Common contest implementation is $O(V^3)$ and returns a maximum matching. Much harder than Kuhn/Hopcroft–Karp; use bipartite algorithms whenever bipartiteness exists.

Typical problem statements.

- Pair as many participants as possible when arbitrary compatibility edges are allowed.
- Maximum domino-like pairing in a non-bipartite conflict graph.

3.6 Tree Center / Diameter

Recognize. You need the diameter, radius, or central vertex/vertices of a tree.

Idea. From any vertex, find a farthest endpoint a ; from a , find a farthest endpoint b . The path $a \rightsquigarrow b$ is a diameter. Its middle vertex (or two middle vertices) is the tree center; with weighted edges, the radius is determined by distances along the diameter. Tree DP can also compute eccentricities for every vertex.

Cost / notes. $O(n)$ with two BFS/DFS for unweighted/nonnegative tree distances, plus path recovery. For negative weighted edges, “farthest from arbitrary” needs care; use tree DP instead.

Typical problem statements.

- Find the minimum possible maximum distance from a chosen facility to all vertices.
- Find the longest path in a tree.
- Choose one/two central vertices minimizing worst-case distance.

3.7 Centroid of a Tree

Recognize. You need a vertex whose removal leaves no connected component larger than $n/2$.

Idea. Compute subtree sizes. Starting from any/root vertex, move to a neighbor whose component after removal has size $> n/2$; when no such neighbor exists, you are at a centroid. A tree has one or two centroids.

Cost / notes. $O(n)$ to compute/find. Do not confuse a *centroid* with the tree center: centroid balances component sizes, center minimizes maximum distance.

Typical problem statements.

- Find a balanced separator vertex of a tree.
- Choose the first split point for centroid decomposition.

3.8 Centroid Decomposition

Recognize. Queries/updates depend on distances or paths to a dynamic set of marked vertices, and ordinary subtree decomposition does not localize them well.

Idea. Choose a centroid, remove it conceptually, recursively decompose each remaining component, and connect component centroids to form a centroid tree of height $O(\log n)$. Every original vertex has $O(\log n)$ centroid ancestors; precompute its distance to each. An update/query aggregates information along those ancestors, often with inclusion–exclusion per child component.

Cost / notes. $O(n \log n)$ preprocessing is typical; point mark/unmark or nearest/count-within-distance queries often cost $O(\log n)$ or $O(\log^2 n)$.

Typical problem statements.

- Toggle red vertices and query distance to the nearest red vertex.
- Count active vertices within distance K of v .
- Count tree paths with a specified length.

3.9 Centroid Tree with Precomputed Distances

Recognize. You specifically want the centroid-decomposition parent tree plus distances from every vertex to each centroid ancestor.

Idea. Build centroid decomposition and, at each recursive level, DFS the current component to append distance-to-current-centroid for its vertices. Store $parentCentroid[c]$ and the per-level distance vector. This turns many online distance queries into a loop over $O(\log n)$ ancestors with no LCA calls.

Cost / notes. The UFMG structure uses $O(n \log n)$ time and memory. It is infrastructure for centroid-decomposition solutions, not a distinct graph-theoretic tree.

Typical problem statements.

- Maintain minimum distance to any active vertex with fast updates.
- Store frequency maps by distance at every centroid to count vertices satisfying distance constraints.

3.10 Dijkstra

Recognize. Single-source shortest paths have nonnegative edge weights.

Idea. Maintain tentative distances and repeatedly finalize the not-yet-finalized vertex with smallest distance using a min-priority queue. Relax outgoing edges. In the common lazy heap implementation, push improved pairs and skip popped entries whose distance is stale.

Cost / notes. $O((V + E) \log V)$ with a binary heap, often written $O(E \log V)$. Fails with negative edges because a “finalized” distance may later improve. 0/1 weights admit 0–1 BFS.

Typical problem statements.

- Shortest travel time with nonnegative road costs.
- Multi-source nearest-facility distances by seeding all sources at distance zero.
- Shortest path in an implicit state graph with positive move costs.

3.11 Dinic / Dinitz Max Flow

Recognize. You need maximum s - t flow in a capacitated directed graph.

Idea. Repeat phases: BFS builds the level graph using residual edges with positive capacity; DFS sends a blocking flow only along edges from level d to $d + 1$. Current-edge pointers prevent rescanning dead edges inside a phase. Residual reverse edges encode cancellation.

Cost / notes. General worst-case $O(V^2 E)$; much faster in typical ICPC inputs, and $O(E\sqrt{V})$ -type bounds for some unit networks. Use 64-bit capacities when sums can overflow.

Typical problem statements.

- Maximum throughput from source to sink.
- Minimum s - t cut via max-flow/min-cut.
- Bipartite matching by a source-left-right-sink flow network.

3.12 Dominator Tree

Recognize. In a directed graph rooted at s , vertex u *dominates* v if every path from s to v passes through u . You need immediate dominators or mandatory checkpoints.

Idea. Lengauer–Tarjan DFS-numbers reachable vertices, computes semidominators using union-find-like link/eval operations, then resolves immediate dominators. The resulting dominator relation is a tree: ancestors of v are exactly the vertices dominating v .

Cost / notes. Near-linear, classically $O((V + E)\alpha)$ with efficient implementation; many contest versions are effectively linear. Only vertices reachable from the root belong to the dominator tree.

Typical problem statements.

- Find vertices that every route from entrance s to target t must pass.
- Determine mandatory states in a control-flow graph.
- Answer dominance queries after preprocessing.

3.13 Euler Path / Euler Cycle

Recognize. You need a walk using every edge exactly once.

Idea. Use Hierholzer: start at the required vertex, repeatedly follow unused edges, and when stuck append the vertex to the answer while backtracking; the reversed output is the Euler trail. Undirected existence: all nonzero-degree vertices connected and 0 or 2 odd degrees. Directed existence uses in/out-degree balance plus appropriate connectivity.

Cost / notes. $O(V + E)$ when edges are removed/marked with adjacency iterators. Distinguish Euler (every edge) from Hamiltonian (every vertex), which is much harder.

Typical problem statements.

- Arrange tickets so every directed flight is used once.
- Traverse every street/edge exactly once and output the route.
- Construct a de Bruijn sequence via an Euler cycle in a de Bruijn graph.

3.14 Floyd–Warshall

Recognize. You need all-pairs shortest paths and V is small/moderate, possibly with negative edges but no relevant negative cycles.

Idea. Dynamic programming over allowed intermediate vertices: for $k = 1..V$, set $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$. After completion, $d[i][j]$ is the shortest path. Boolean OR/AND gives transitive closure; max-min variants solve bottleneck paths.

Cost / notes. $O(V^3)$ time, $O(V^2)$ memory. A negative $d[k][k]$ indicates a negative cycle reachable through k . Guard infinity additions against overflow.

Typical problem statements.

- Answer many shortest-path queries on a graph with $n \leq 500$.
- Compute transitive closure/reachability.
- Find minimax/bottleneck path values between every pair.

3.15 Functional Graph

Recognize. Every vertex has exactly one outgoing edge, so each weak component is one directed cycle with directed trees feeding into it.

Idea. Exploit this structure rather than general graph machinery: DFS/coloring or indegree peeling identifies cycles; binary lifting answers k -th successor; distances/meeting queries decompose into tree-to-cycle depth plus movement around the cycle. Reverse edges represent the in-trees.

Cost / notes. Cycle decomposition is $O(n)$; binary lifting is $O(n \log K)$ preprocessing and $O(\log K)$ per jump. Specialized distance queries can be $O(\log n)$.

Typical problem statements.

- Where are you after k repeated applications of function f ?
- Find cycle length and distance-to-cycle for every node.
- Minimum steps for one functional-graph node to reach another.

3.16 HLD – Edge Values

Recognize. A static tree has edge weights and you need online path queries/updates (sum/max/min, point/range changes).

Idea. Heavy-Light Decomposition chooses each node’s largest-subtree child as heavy, forming $O(\log n)$ heavy paths along any root path. Map edges to the position of their deeper endpoint in a base array and use a segment tree. Decompose path u – v into heavy-chain intervals, excluding the LCA position for edge-valued queries.

Cost / notes. $O(n)$ decomposition; typically $O(\log^2 n)$ per path query/update with a segment tree, reducible in some variants. Edge/vertex indexing at the LCA is the main difference from vertex HLD.

Typical problem statements.

- Maximum edge weight on path u – v with edge updates.
- Add to all edges on a path and query path sum.

3.17 HLD – Vertex Values

Recognize. A static tree has values on vertices and online path/subtree queries or updates are required.

Idea. Build heavy chains and assign each vertex a position in decomposition order. A path is split into $O(\log n)$ contiguous chain segments queried in a segment tree/Fenwick tree. For vertex values, include the LCA position. A DFS order can simultaneously make each subtree contiguous.

Cost / notes. Usually $O(\log^2 n)$ per path operation and $O(\log n)$ per subtree segment operation after $O(n)$ preprocessing.

Typical problem statements.

- Sum values on path $u-v$ under point updates.
- Add to all vertices on a path and query maximum.
- Combine path and subtree operations on a rooted tree.

3.18 HLD without Updates

Recognize. Tree path queries are static; you want HLD decomposition but can replace dynamic segment trees with faster static structures/prefixes.

Idea. Decompose into heavy paths as usual. Since values never change, store prefix aggregates or static RMQ/sparse-table data per chain. A path still crosses $O(\log n)$ chains, but each chain fragment can often be answered in $O(1)$.

Cost / notes. Typically $O(n \log n)$ or $O(n)$ preprocessing depending on the per-chain structure, and $O(\log n)$ path query. Particularly useful for non-invertible static aggregates where simple root prefix subtraction is impossible.

Typical problem statements.

- Static maximum/minimum edge on arbitrary tree paths.
- Many path-gcd queries with no updates.

3.19 Hopcroft–Karp

Recognize. You need maximum-cardinality matching in a large bipartite graph.

Idea. Alternate BFS and DFS phases. BFS layers the graph of alternating paths from all free left vertices and finds the minimum augmenting-path length; DFS then finds a maximal set of vertex-disjoint augmenting paths of that length simultaneously. Flip all found paths.

Cost / notes. $O(E\sqrt{V})$ worst-case and usually very fast. After maximum matching, König’s theorem lets you recover a minimum vertex cover / maximum independent set in a bipartite graph.

Typical problem statements.

- Match workers to compatible jobs.
- Maximum number of nonconflicting row-column pairings.
- Minimum bipartite vertex cover after matching.

3.20 Tree Isomorphism

Recognize. You need to decide whether two unlabeled trees have the same structure up to renaming vertices, or canonicalize rooted/unrooted trees.

Idea. For rooted trees, recursively canonicalize each vertex from the multiset of child canonical forms, e.g. sort child IDs/hashes and intern the vector. For unrooted trees, find the one/two centers and compare rooted canonical forms at the centers. Random hashing is faster but probabilistic; integer interning is collision-free.

Cost / notes. $O(n \log n)$ straightforward due to sorting child signatures, often near $O(n)$ with optimized sorting/interning. For forests, canonicalize components and sort them.

Typical problem statements.

- Determine whether two tree-shaped molecules/networks are structurally identical.
- Group many trees into isomorphism classes.
- Test whether a tree has a symmetry after rerooting.

3.21 Johnson’s Algorithm

Recognize. You need all-pairs shortest paths in a sparse graph that may have negative edges but no negative cycles.

Idea. Add a super-source and run Bellman–Ford to obtain potentials $h[v]$. Reweight each edge to $w'(u, v) = w(u, v) + h[u] - h[v] \geq 0$, preserving shortest-path ordering. Run Dijkstra from every source, then convert distances back using the potentials.

Cost / notes. $O(VE + VE \log V)$ with a binary heap after one Bellman–Ford, usually summarized $O(VE \log V)$ for sparse graphs. Detect negative cycles during the Bellman–Ford phase.

Typical problem statements.

- All-pairs shortest paths on a sparse graph with some negative edges.
- Many source-target shortest-path queries where Floyd–Warshall is too cubic.

3.22 Kosaraju SCC

Recognize. You need strongly connected components of a directed graph.

Idea. First DFS the original graph and record vertices by finishing time. Then process vertices in reverse finishing order on the transposed graph; every DFS reached in this second pass is one SCC. Contract SCCs to obtain a DAG.

Cost / notes. $O(V + E)$ time and memory, but requires both graph and reverse graph (or equivalent traversal). Iterative DFS may be safer for very deep graphs.

Typical problem statements.

- Compress a directed graph into its SCC condensation DAG.
- Determine which vertices are mutually reachable.
- Solve 2-SAT using SCCs of the implication graph.

3.23 Kruskal

Recognize. You need a minimum spanning tree/forest of an undirected weighted graph.

Idea. Sort edges by nondecreasing weight. Scan them; add an edge iff its endpoints currently lie in different DSU components, then union those components. The cut property guarantees every accepted edge can belong to an MST.

Cost / notes. $O(E \log E)$ for sorting plus near-linear DSU work. For maximum spanning tree, reverse the order. The selected forest also supports bottleneck/minimax reasoning.

Typical problem statements.

- Cheapest set of roads connecting all cities.
- Maximum spanning tree for widest-path transformations.
- Cluster points by stopping Kruskal after a target number of components.

3.24 Kuhn Bipartite Matching

Recognize. You need maximum bipartite matching and constraints are small/moderate enough for the simple DFS augmenting-path method.

Idea. For each left vertex, DFS through its neighbors searching for an unmatched right vertex or recursively rerouting the current owner. On success, flip the augmenting path. Reset visitation per search (or use timestamps). Random/order heuristics can improve practical performance.

Cost / notes. Worst-case $O(VE)$, commonly $O(V_L E)$. Simpler than Hopcroft–Karp and often ideal up to a few thousand vertices/edges.

Typical problem statements.

- Assign each applicant at most one acceptable job.
- Maximum rook placement after reducing board segments to bipartite matching.

3.25 LCA with Binary Lifting

Recognize. You need lowest common ancestors, ancestor jumps, or path distances in a static rooted tree.

Idea. DFS computes depth and $up[0][v] = parent(v)$. Precompute $up[k][v] = up[k-1][up[k-1][v]]$. To find LCA, first lift the deeper vertex to equal depth, then lift both from highest power downward while their ancestors differ.

Cost / notes. $O(n \log n)$ preprocessing/memory and $O(\log n)$ per LCA/ k -th ancestor. Distance is $depth[u] + depth[v] - 2depth[lca]$ for unit edges (or root-distance analog for weighted).

Typical problem statements.

- Distance between arbitrary tree vertices.
- Find the k -th ancestor of a node.
- Test whether a is an ancestor of b .

3.26 LCA with HLD

Recognize. You already have Heavy-Light Decomposition, or you want LCA with linear memory and logarithmic chain jumps.

Idea. While chain heads differ, move upward from the vertex whose chain head is deeper to the parent of that head. Once both vertices share a chain, the shallower one is the LCA. This is the same chain

decomposition used for path queries.

Cost / notes. $O(n)$ HLD preprocessing and $O(\log n)$ LCA because any root path crosses at most $O(\log n)$ light edges/chains. No binary-lifting table is required.

Typical problem statements.

- Compute LCA inside an existing HLD path-query solution.
- Save memory versus an $n \log n$ ancestor table while keeping logarithmic LCA.

3.27 LCA with RMQ

Recognize. You need many LCA queries on a static tree and want constant-time queries after preprocessing.

Idea. Euler-tour the rooted tree, recording every visit and its depth. Between the first occurrences of u and v , the vertex of minimum Euler depth is their LCA. Thus LCA reduces to static RMQ; use sparse table for $O(1)$ queries with $O(n \log n)$ preprocessing, or a linear RMQ structure for optimal bounds.

Cost / notes. Euler tour length is $2n - 1$. With sparse table: $O(n \log n)$ preprocessing/memory, $O(1)$ query; with Farach-Colton–Bender: $O(n)$ preprocess/memory and $O(1)$ query.

Typical problem statements.

- Huge number of static LCA queries.
- Convert tree ancestor queries into an array RMQ problem.

3.28 Line Tree

Recognize. You need minimum-edge/bottleneck queries between vertices, and the UFMG “Line Tree” reduction is available.

Idea. The notebook sorts edges descending and effectively builds the maximum spanning forest while merging components into linearized structures, reducing path-minimum queries to RMQ/segment-tree queries. If the input is not already a tree, the relevant path is in the maximum spanning tree; its minimum edge equals the maximum possible bottleneck between the pair.

Cost / notes. Notebook: $O(n \log n)$ build and $O(\log n)$ query (with edge count assumptions of the implementation). This is a specialized bottleneck/min-on-tree structure, not the graph-theory line graph.

Typical problem statements.

- For u, v , find the largest possible minimum edge weight along any path.
- Query the minimum edge on many paths of a fixed weighted tree.

3.29 Link–Cut Tree

Recognize. You maintain a dynamic forest under **link**, **cut**, connectivity, rerooting and path aggregate/update queries.

Idea. Represent each preferred path by a splay tree. **access**(v) repeatedly detaches/reattaches preferred right subtrees so the path from the represented-tree root to v becomes one auxiliary splay path. **makeroot**(v) exposes then lazily reverses this path. Link/cut and path queries reduce to a few access/splay operations.

Cost / notes. $O(\log n)$ amortized per operation. Very implementation-sensitive: push reverse/lazy tags before rotations and maintain auxiliary-tree aggregates exactly.

Typical problem statements.

- Dynamically add/remove forest edges and query sum/max on path $u-v$.
- Maintain a dynamic MST under edge insertions using max-edge-on-path replacement.

3.30 Link–Cut Tree – Edge Values

Recognize. The dynamic forest has data on edges rather than vertices.

Idea. Common transformation: represent every original edge by an extra LCT node connected to its two endpoint vertices; store the edge weight/value on that node and neutral values on vertex nodes. Then normal vertex-path aggregate operations automatically aggregate edges. Alternatively, specialized edge bookkeeping can exclude endpoints.

Cost / notes. $O(\log n)$ amortized per link/cut/path operation, with $n + m$ LCT nodes if edge-nodes are used.

Typical problem statements.

- Dynamic forest with edge-weight updates and maximum edge on a path.
- Online MST replacement: find/remove the heaviest edge on the cycle created by a new edge.

3.31 Link–Cut Tree – Vertex Values

Recognize. The dynamic forest has mutable values attached to vertices and path aggregates/updates are required.

Idea. Use standard LCT nodes directly for original vertices. After **makeroot**(u); **access**(v), the auxiliary splay rooted at v represents exactly the $u-v$ path, so its stored aggregate is the answer and a lazy tag can update the path.

Cost / notes. $O(\log n)$ amortized per operation. Subtree queries are not as natural as path queries and require extra virtual-subtree bookkeeping.

Typical problem statements.

- Link/cut edges while querying XOR/sum of vertex values on paths.
- Add a value to every vertex on a dynamically changing tree path.

3.32 Max Flow with Lower Bounds

Recognize. Every directed edge requires $L_e \leq f_e \leq U_e$ rather than just $0 \leq f_e \leq U_e$, and you need feasibility or an $s-t$ flow.

Idea. Pre-send the lower bound L_e , leaving residual capacity $U_e - L_e$. This creates vertex balance demands: incoming lower flow minus outgoing lower flow. Add super-source SS to deficit/supply nodes and super-sink TT accordingly; a max flow saturating all SS edges proves a feasible circulation. For $s-t$ flow, add an auxiliary $t \rightarrow s$ infinite edge, find a feasible circulation, then optimize further in the residual graph.

Cost / notes. One or a constant number of ordinary max-flow runs on $O(V + E)$ extra structure. Recover original flow by adding lower bounds back.

Typical problem statements.

- Find a feasible network flow where each pipe has minimum and maximum throughput.
- Assign quantities with per-edge lower quotas.
- Maximum $s-t$ flow subject to mandatory minimum flows.

3.33 Min-Cost Max-Flow

Recognize. You need maximum/required flow while also minimizing total edge cost, with capacities and possibly negative costs.

Idea. Repeatedly augment flow along the cheapest residual $s-t$ path. Successive shortest augmenting path with vertex potentials reweights residual costs to nonnegative values so Dijkstra can be used after an initial Bellman–Ford/SPFA/potential initialization. Update potentials by shortest distances and send the bottleneck flow.

Cost / notes. Roughly $O(FE \log V)$ if augmenting one unit at a time, or $O(AE \log V)$ for A augmentations sending bottlenecks; cost-scaling variants are faster. Reverse residual edges carry negative cost.

Typical problem statements.

- Assign workers to jobs minimizing total cost with capacities.
- Send K units through a network at minimum cost.
- Minimum-cost bipartite matching.

3.34 Prüfer Code

Recognize. You need a bijection between labeled trees on n vertices and sequences of length $n - 2$, or need to count/generate labeled trees.

Idea. Encoding repeatedly removes the smallest leaf and outputs its neighbor. Decoding starts from degrees $1 + \text{frequency}(v)$ in the sequence, repeatedly connects the smallest current leaf to the next code value, then connects the final two leaves. With pointer tricks, both can be linear for suitable label order; a heap gives $O(n \log n)$.

Cost / notes. Sequence length exactly $n - 2$ and Cayley’s formula n^{n-2} follows immediately. Degree of vertex v is $1 +$ its number of appearances in the code.

Typical problem statements.

- Count labeled trees with specified degree constraints.
- Encode/decode a labeled tree.
- Generate random labeled trees by sampling Prüfer sequences.

3.35 Rooted Euler Tour Tree

Recognize. You need dynamic rooted-forest operations where a rooted tree/subtree is represented as a sequence and cuts/links/reroot-like operations can be expressed by sequence splits/joins.

Idea. Represent DFS/Euler occurrences in a balanced BST (often implicit treap/splay). For a rooted variant, subtree occurrences can be contiguous, so splitting the sequence isolates a subtree; linking concatenates it at the appropriate occurrence. Aggregates/lazy tags live on the sequence tree.

Cost / notes. Typically $O(\log n)$ expected/amortized per split/join/link/cut and sequence aggregate, depending on the balanced BST. Compared with LCT, Euler-tour trees are especially natural for component/subtree aggregates.

Typical problem statements.

- Dynamic rooted forest with subtree-sum queries under cut/link.
- Maintain component aggregates as rooted trees are attached/detached.

3.36 Sack / DSU on Tree

Recognize. For every tree vertex, you need an answer about the multiset of values/colors in its subtree, and updates to that multiset are cheap but clearing/rebuilding is expensive.

Idea. Process all light children and discard their data; process the heavy child and keep its data. Then add every vertex from the light subtrees into the heavy child's retained structure, answer for the current node, and discard only when requested by the parent. Each vertex is re-added only $O(\log n)$ times because moving through a light edge at least doubles subtree size.

Cost / notes. Usually $O(n \log n)$ if each add/remove is $O(1)$, or multiplied by the data-structure operation cost. This is offline/static-tree technique, unlike DSU union-find.

Typical problem statements.

- For each node, count distinct colors in its subtree.
- Find the most frequent color/sum of modes in every subtree.
- Count subtree pairs/values satisfying a frequency condition.

3.37 Stable Marriage (Gale–Shapley)

Recognize. Two equally sized sides each rank the other side, and you need a stable perfect matching: no unmatched pair prefers each other over their assigned partners.

Idea. One side proposes in preference order. A receiver keeps the best proposal seen and rejects any worse one; rejected proposers continue down their lists. Once nobody is free with untried choices, the matching is stable. The result is proposer-optimal and receiver-pessimal among stable matchings.

Cost / notes. $O(n^2)$ proposals/time with rank arrays for $O(1)$ preference comparison, $O(n^2)$ input memory. Variants handle incomplete lists/unmatched participants.

Typical problem statements.

- Match students to partners under strict mutual rankings with no blocking pair.
- Stable assignment of residents to positions in the one-to-one setting.

3.38 Tarjan SCC

Recognize. You need strongly connected components in one DFS pass, often with low memory and no transpose graph.

Idea. DFS assigns discovery index and low-link. Keep active vertices on a stack; after exploring edges, if $\text{low}[v] = \text{index}[v]$, v is the root of an SCC, so pop until v . Only edges to vertices still on the active stack affect SCC low-link.

Cost / notes. $O(V + E)$ time and memory. Do not mix the SCC low-link condition with the undirected bridge/articulation low-link rules; they look similar but handle edges differently.

Typical problem statements.

- Compress a directed graph into SCCs.
- Solve 2-SAT.
- Detect mutually reachable groups in one DFS traversal.

3.39 Top Tree

Recognize. You need a very general dynamic-tree structure supporting link/cut plus path and sometimes whole-component/subtree aggregates with composable cluster information.

Idea. A top tree recursively groups tree edges into clusters with at most two boundary vertices. Expose operations rearrange clusters so the requested path/component becomes the top cluster; cluster merge formulas maintain the desired DP/aggregate. Modern implementations often realize the cluster hierarchy with splay-like rotations.

Cost / notes. $O(\log n)$ amortized per dynamic-tree operation in standard top-tree frameworks, with problem-specific $O(1)$ cluster merge. Much more complex than LCT, but can maintain DPs that are awkward in preferred-path representations.

Typical problem statements.

- Maintain tree diameter dynamically under link/cut and edge-weight changes.
- Dynamic forest DP where each cluster answer depends only on its one/two boundary states.

3.40 Topological Sort

Recognize. You need a linear ordering of a directed acyclic graph (DAG) where every edge goes from earlier to later, or need to detect a directed cycle.

Idea. Kahn's algorithm repeatedly removes indegree-zero vertices, decrementing neighbors; if fewer than V vertices are removed, a cycle exists. DFS postorder reversed is another method. Lexicographically smallest topological order uses a min-heap of current indegree-zero vertices.

Cost / notes. $O(V + E)$ with a queue; $O((V + E) \log V)$ with a priority queue for lexicographic order. A DAG may have many valid orders.

Typical problem statements.

- Schedule tasks subject to prerequisites.
- DP over a DAG in dependency order.
- Determine whether constraints u before v contain a cycle.

3.41 Unrooted Euler Tour Tree

Recognize. You need fully dynamic undirected forests and component-level aggregates/connectivity using an Euler-tour sequence, without fixing a permanent root.

Idea. Represent each undirected tree's Euler tour as a cyclic sequence in a balanced BST, often with directed edge occurrences. Rerooting is a cyclic rotation (split + merge); link splices two tours with new edge occurrences; cut removes the pair and separates the sequence into two tours.

Cost / notes. Typically $O(\log n)$ expected/amortized per reroot/link/cut/connectivity/component aggregate. Unlike LCT, path aggregates are less direct; component aggregates are very natural.

Typical problem statements.

- Fully dynamic forest connectivity under edge insertions/deletions.
- Maintain sum/size of each tree component while linking and cutting edges.

3.42 Exact Minimum Vertex Cover (General Graph)

Recognize. You need a minimum vertex cover in a small/medium general graph where polynomial-time bipartite König does not apply.

Idea. Branch on a high-degree vertex v : either include v in the cover, or exclude it, which forces every neighbor of v into the cover. Apply reductions for paths/cycles/low-degree components and recurse on the remaining induced graph. Bitsets make component and neighborhood operations fast.

Cost / notes. The photographed UFMG implementation targets about $O(n 1.38^n)$ and notes practical use around $n \approx 90$ for favorable instances. This is exponential exact search, not the bipartite matching reduction.

Typical problem statements.

- Find the smallest set of vertices touching every edge for $n \lesssim 80\text{--}100$.
- Solve maximum independent set via $\alpha(G) = n - \tau(G)$ using exact vertex cover.

3.43 Virtual Tree

Recognize. Each query marks only k important vertices of a large static tree, and the answer depends on paths among them; processing all n vertices per query is too slow.

Idea. Sort marked vertices by Euler entry time, add LCAs of adjacent marked vertices, sort/unique again, and connect nodes with a stack by ancestor relation. The resulting virtual tree contains $O(k)$ vertices and preserves all relevant ancestor/path structure; edge weights become original-tree distances between virtual parent/child. Run the query DP only on this tree.

Cost / notes. After $O(n \log n)$ LCA preprocessing, each query is $O(k \log k)$ (or near $O(k)$ if already ordered). Remember to add LCAs and clear only touched virtual adjacency.

Typical problem statements.

- Given k special vertices, compute total length of the minimal connecting subtree.
- Per query, DP over only marked nodes and their LCAs.
- Count/optimize pairwise relations among a small queried subset of tree vertices.

4 Mathematics

4.1 2-SAT

Recognize. Each Boolean clause has at most two literals, e.g. $(a \vee \neg b)$, and you need feasibility plus an assignment.

Idea. Convert clause $(x \vee y)$ into implications $(\neg x \rightarrow y)$ and $(\neg y \rightarrow x)$. Build a graph on one node per literal and compute SCCs. The formula is impossible iff some variable and its negation belong to the same SCC; otherwise SCC topological order yields a satisfying assignment.

Cost / notes. $O(V + E)$ where $V = 2n$ and $E = 2m$. Many “choose one of two options” constraints translate directly: $x \Rightarrow y$, not-both, at-least-one, equality/inequality.

Typical problem statements.

- Choose one of two meeting times per event while satisfying pairwise conflicts.
- Assign each variable true/false under clauses of size two.
- Orient binary choices so forbidden pairs never occur together.

4.2 Interpolation Evaluation at Consecutive Points

Recognize. You know $P(0), P(1), \dots, P(n-1)$ for a degree $< n$ polynomial and need $P(x)$, often modulo a prime and with huge x .

Idea. Use Lagrange interpolation, but exploit consecutive $x_i = i$. Precompute factorials/inverse factorials and prefix/suffix products of $(x - i)$. Each Lagrange basis denominator is $i!(-1)^{n-1-i}(n-1-i)!$, so all n terms are evaluated in linear time.

Cost / notes. $O(n)$ per evaluation after factorial preparation, $O(n)$ memory. Requires invertible denominators in the modulus (typically prime modulus with $n < \text{mod}$). If $x \in [0, n)$, simply return the known sample.

Typical problem statements.

- A counting function is known to be a degree- d polynomial; compute it at $N \gg d$.
- Evaluate a polynomial from its values at $0, 1, \dots, d$ without recovering coefficients.

4.3 Berlekamp–Massey

Recognize. You have initial terms of a sequence over a field and suspect it satisfies a short linear recurrence; you need the shortest recurrence and possibly a huge-index term.

Idea. BM scans terms and maintains the smallest connection polynomial consistent so far. A nonzero discrepancy means the current recurrence fails; update it using a shifted/scaled previous successful polynomial. About $2L$ correct terms determine a recurrence of order L . Then compute term k by polynomial exponentiation modulo the characteristic polynomial (Kitamasa/Bostan-like evaluation).

Cost / notes. Recurrence discovery $O(n^2)$. Standard k -th-term evaluation is $O(L^2 \log k)$; NTT-based polynomial methods can reduce it toward $O(L \log L \log k)$. Arithmetic must be over a field.

Typical problem statements.

- Infer a hidden recurrence from the first few hundred terms and compute term 10^{18} .
- Accelerate a DP sequence known empirically/theoretically to have a small linear recurrence.

4.4 Binomial Distribution

Recognize. There are n independent Bernoulli trials with equal success probability p , and you need the probability of exactly/at most/at least k successes.

Idea. For exactly k , use $\Pr[X = k] = \binom{n}{k} p^k (1-p)^{n-k}$. For numerical stability at large n , compute logarithms using precomputed $\log(n!)$ and exponentiate, or derive adjacent probabilities by a multiplicative recurrence when summing a CDF.

Cost / notes. Exact single probability is $O(1)$ after $O(N)$ log-factorial preprocessing; a range/CDF is

usually linear in the number of terms unless a special-function library/approximation is used. Handle $p = 0, 1$ separately.

Typical problem statements.

- Probability that exactly k of n independent attempts succeed.
- Probability of passing when at least k answers are independently correct with probability p .

4.5 GCD / LCM Convolution

Recognize. You need convolution where indices combine by gcd or lcm rather than by addition: e.g. $h[k] = \sum_{\text{gcd}(i,j)=k} f[i]g[j]$.

Idea. Use zeta transforms on the divisibility poset. For gcd convolution, transform by summing over multiples; for lcm convolution, sum over divisors. After transforming, multiply pointwise, then Möbius-invert using prime/sieve-style loops. This is the divisibility analogue of FFT turning convolution into pointwise multiplication.

Cost / notes. Typically $O(N \log N)$ -ish harmonic complexity, concretely $O(N \log N)$ from divisor/multiple loops, and $O(N)$ memory. The exact transform direction for gcd vs lcm is easy to swap accidentally.

Typical problem statements.

- Count ordered pairs whose gcd equals each k .
- Combine arithmetic-function arrays under lcm convolution.
- Compute pair statistics by divisor relation for all values simultaneously.

4.6 Coprime Basis

Recognize. You have several integers and want a shared factor basis whose basis elements are pairwise coprime, without fully prime-factorizing every input.

Idea. Maintain basis factors. When a new factor shares gcd $g > 1$ with an existing basis element, split the conflicting numbers by g and continue until all surviving basis elements are pairwise coprime. Then every input can be represented as a product $\prod B_j^{e_j}$ by repeated division.

Cost / notes. The UFMG structure uses repeated gcd/splitting; its expected bound is roughly $O(n(n \log \log M + \log^2 M))$ for n inserts and maximum value M , with a small bound on how often integers can split. Useful when gcd is cheap but full factorization is undesirable.

Typical problem statements.

- Express many related composite numbers in a common independent factor basis.
- Compare exponent vectors of numbers when only pairwise gcd information is easy to obtain.

4.7 Sieve of Eratosthenes / Linear Sieve

Recognize. You need primes, smallest prime factors, Möbius μ , Euler φ , or fast factorization for every integer up to N .

Idea. Classic sieve marks multiples of each prime starting at p^2 . Linear sieve keeps the list of primes and, for each i , visits $i \cdot p$ only until p exceeds the smallest prime factor of i , so every composite is generated once. Extra multiplicative functions can be updated during this transition.

Cost / notes. Classic sieve $O(N \log \log N)$, linear sieve $O(N)$, both $O(N)$ memory. For only primality up to large N , bit-packed odd-only sieves save memory.

Typical problem statements.

- List/count primes up to N .
- Factor many numbers $\leq N$ quickly using smallest prime factor.
- Precompute $\mu(n)$ or $\varphi(n)$ for divisor-sum problems.

4.8 Cycle Detection – Tortoise and Hare

Recognize. You repeatedly apply a deterministic function $x_{t+1} = f(x_t)$ and need to detect the eventual cycle using $O(1)$ memory.

Idea. Floyd uses a tortoise moving one step and a hare moving two until they meet inside the cycle. Reset one pointer to the start and move both one step to find the cycle entry μ ; keep one fixed and loop the other to get cycle length λ . Brent’s algorithm is an alternative with fewer function calls.

Cost / notes. $O(\mu + \lambda)$ time and $O(1)$ memory. It detects cycles in the generated sequence, not arbitrary graph cycles.

Typical problem statements.

- Find the preperiod and period of a pseudorandom recurrence.
- Detect repetition in a functional graph without storing visited states.
- Pollard-rho style iteration needs a cycle-detection mechanism.

4.9 Division Trick / Quotient Grouping

Recognize. A loop involves $\lfloor n/i \rfloor$ for all $i \leq n$, but n is too large to iterate linearly.

Idea. The quotient $q = \lfloor n/i \rfloor$ is constant on an interval. At left endpoint l , compute $q = n/l$ and the maximal right endpoint $r = \lfloor n/q \rfloor$; process the whole block $[l, r]$ at once and continue with $l = r + 1$. There are only $O(\sqrt{n})$ distinct quotients.

Cost / notes. $O(\sqrt{n})$ blocks/time when each block is $O(1)$. This trick appears constantly in divisor summatory functions and floor-sum formulas.

Typical problem statements.

- Compute $\sum_{i=1}^n \lfloor n/i \rfloor$ in $O(\sqrt{n})$.
- Group all distinct values of n/i for a huge n .
- Speed up arithmetic sums whose summand depends only on $\lfloor n/i \rfloor$.

4.10 Linear Diophantine Equation

Recognize. You need integer solutions to $ax + by = c$ or bounded/counting variants.

Idea. A solution exists iff $g = \gcd(a, b)$ divides c . Extended Euclid gives x_0, y_0 with $ax_0 + by_0 = g$; multiply by c/g . All solutions are $x = x_0 + (b/g)t$, $y = y_0 - (a/g)t$. Intersect the allowed range for integer t to handle bounds/nonnegativity.

Cost / notes. $O(\log \min(|a|, |b|))$ to obtain one solution, then $O(1)$ arithmetic to parameterize/count bounded solutions. Carefully implement floor/ceil division for negative numbers.

Typical problem statements.

- Find nonnegative counts x, y satisfying $ax + by = c$.
- Solve a modular linear equation $ax \equiv c \pmod{m}$.
- Count integer lattice points on a line segment $ax + by = c$ within bounds.

4.11 Extended Euclidean Algorithm

Recognize. You need $\gcd(a, b)$ together with coefficients x, y such that $ax + by = \gcd(a, b)$.

Idea. Run Euclid recursively/iteratively and back-substitute coefficients. The returned Bézout coefficients immediately solve linear Diophantine equations and give modular inverses when $\gcd(a, m) = 1$: $a^{-1} \equiv x \pmod{m}$.

Cost / notes. $O(\log \min(|a|, |b|))$ time and $O(1)$ iterative memory. Normalize gcd/sign and modular representatives if later formulas assume nonnegative values.

Typical problem statements.

- Compute modular inverse modulo a non-prime m .
- Solve $ax + by = c$.
- Merge congruences in generalized CRT when moduli are not coprime.

4.12 Fast Exponentiation

Recognize. You need a^e , matrix powers, permutation powers, or repeated application of any associative operation for huge exponent e .

Idea. Binary exponentiation writes e in binary. Repeatedly square the base; whenever the current bit is one, multiply it into the answer. The same pattern works for any monoid: matrices, transformations, polynomials modulo another polynomial, etc.

Cost / notes. $O(\log e)$ multiplications and $O(1)$ extra state beyond the objects. For modular multiplication, reduce after every product and avoid overflow.

Typical problem statements.

- Compute $a^{10^{18}} \bmod m$.
- Compute the k -th step of a linear recurrence using matrix exponentiation.
- Apply a permutation or transition operator k times.

4.13 Fast Walsh–Hadamard Transform (FWHT)

Recognize. You need convolution over bitwise XOR, AND, or OR instead of ordinary index addition.

Idea. Apply butterfly transforms over each bit. XOR uses $(a, b) \mapsto (a + b, a - b)$ and inverse divides by 2^B ; OR/AND use triangular zeta-style butterflies with subtraction for the inverse. Transform both arrays, multiply pointwise, inverse-transform.

Cost / notes. $O(B2^B) = O(N \log N)$ time and $O(N)$ memory. Choose the transform matching the convolution definition; modular XOR inverse requires 2 invertible.

Typical problem statements.

- Count pairs with $a \oplus b = k$ for every k .
- Compute OR/AND convolution of mask-frequency arrays.
- DP/counting over subsets where combination is bitwise XOR.

4.14 FFT

Recognize. You need fast convolution/polynomial multiplication over integers/reals/complex numbers, with sizes too large for $O(n^2)$.

Idea. Evaluate coefficients at roots of unity using divide-and-conquer Cooley–Tukey butterflies, multiply pointwise, then inverse transform. Pad to a power of two at least $n + m - 1$. For integer convolution, round carefully; split coefficients or use multiple transforms when precision is tight.

Cost / notes. $O(N \log N)$ time and $O(N)$ memory. Complex floating error is the main hazard; NTT is exact when a suitable modulus exists.

Typical problem statements.

- Multiply two large polynomials.
- Count pair sums/differences for all values by convolving frequency arrays.
- Multiply huge integers represented in base chunks.

4.15 Gaussian Elimination

Recognize. You need to solve a linear system, determine rank/consistency, invert a matrix, or compute determinant over a field/reals.

Idea. Process columns and choose a pivot row with nonzero (or largest absolute) pivot. Swap it into place, normalize if convenient, and eliminate the variable from other rows (Gauss–Jordan) or rows below (Gaussian elimination + back substitution). Track pivot columns to identify free variables.

Cost / notes. $O(nm \min(n, m))$, $O(n^3)$ for an $n \times n$ system. Over floating point use partial pivoting and an epsilon; modulo prime use modular inverses; composite moduli are not fields.

Typical problem statements.

- Solve n linear equations in n unknowns.
- Find matrix rank/determinant/inverse.
- Determine whether geometric/algebraic constraints have zero, one, or infinitely many solutions.

4.16 Gaussian Elimination over $\mathbb{F}_2$ (Gauss–Z2)

Recognize. Equations use XOR/parity, so coefficients and variables are bits.

Idea. Run Gaussian elimination with addition replaced by XOR. Store each row as a machine-word bitset, choose a row containing the pivot bit, swap, and XOR it into rows that need elimination. The augmented column gives consistency/solutions; pivot rows also form a linear XOR basis.

Cost / notes. With bitsets, roughly $O(n^3/w)$ for an n -variable dense system, often very fast in C++. No division is needed because the only nonzero coefficient is 1.

Typical problem statements.

- Solve a system of parity/XOR constraints.
- Lights Out and switch puzzles.
- Compute rank/number of solutions of binary vectors.

4.17 Numerical Integration

Recognize. You need a numeric approximation of $\int_a^b f(x) dx$ when no convenient closed form is available and f is sufficiently smooth.

Idea. Simpson’s rule approximates on an interval using endpoint and midpoint samples; adaptive Simpson recursively compares one coarse Simpson estimate to the sum of two half-interval estimates and subdivides until the error estimate is small enough. Other quadratures follow the same sample-and-refine idea.

Cost / notes. No fixed combinatorial bound: work depends on smoothness and tolerance. Adaptive

Simpson is excellent for smooth functions but must cap recursion/handle singularities, discontinuities and infinite domains explicitly.

Typical problem statements.

- Compute area under a smooth curve to 10^{-9} error.
- Integrate a geometric cross-section function when the antiderivative is awkward.

4.18 Modular Inverse

Recognize. You need $a^{-1} \bmod m$, i.e. $ax \equiv 1 \pmod m$.

Idea. An inverse exists iff $\gcd(a,m) = 1$. Use extended Euclid for arbitrary modulus. If m is prime and $a \not\equiv 0$, Fermat gives a^{m-2} . For all inverses 1.. n under prime m , the recurrence $inv[i] = m - (m/i)inv[m \bmod i] \bmod m$ is linear-time.

Cost / notes. $O(\log m)$ per inverse by Euclid/power; $O(n)$ for a table. Never “divide modulo m ” unless the divisor is invertible.

Typical problem statements.

- Compute combinations/factorial ratios modulo a prime.
- Solve modular linear equations.
- Normalize rational expressions in modular arithmetic.

4.19 Karatsuba Multiplication

Recognize. You need to multiply large integers/polynomials faster than quadratic time but an FFT/NTT is unnecessary or inconvenient.

Idea. Split each operand into low/high halves: $A = A_0 + A_1B$, $C = C_0 + C_1B$. Compute only three recursive products $z_0 = A_0C_0$, $z_2 = A_1C_1$, and $z_1 = (A_0 + A_1)(C_0 + C_1) - z_0 - z_2$, then recombine.

Cost / notes. $T(n) = 3T(n/2) + O(n) = O(n^{\log_2 3}) \approx O(n^{1.585})$. Use schoolbook multiplication below a threshold; padding/carry handling affects practical performance.

Typical problem statements.

- Multiply big integers with a few thousand digits.
- Multiply medium-size polynomials when FFT precision/setup is overkill.

4.20 Discrete Logarithm

Recognize. Given a, b, m , find x such that $a^x \equiv b \pmod m$, usually in a multiplicative group.

Idea. Baby-step giant-step writes $x = iq + j$ with $q \approx \sqrt{M}$. Store baby powers a^j and walk giant steps using a^{-q} until a collision with b -related values appears. For non-coprime generalized cases, repeatedly remove gcd factors before applying BSGS.

Cost / notes. $O(\sqrt{M})$ time and memory for group/order size M ; hash tables give expected constant lookup. Existence is not guaranteed, and modular inverses require coprimality in the standard version.

Typical problem statements.

- Find the exponent x in $g^x \equiv h \pmod p$.
- Solve repeated-state exponent equations in a cyclic modular group.

4.21 Miller–Rabin Primality Test

Recognize. You need fast primality testing for large 64-bit integers where sieving to $\sqrt{n}$ is impossible.

Idea. Write $n - 1 = d2^s$ with d odd. For each witness a , compute $x = a^d \bmod n$; n passes that witness if $x = 1$ or repeated squaring reaches $n - 1$. Otherwise a proves compositeness. A fixed small witness set makes the test deterministic on unsigned 64-bit ranges.

Cost / notes. $O(k \log n)$ modular multiplications/exponentiation steps for k witnesses (more precisely $O(k \log^3 n)$ bit complexity with naive arithmetic). Handle small primes/even numbers first and use overflow-safe multiplication.

Typical problem statements.

- Test whether arbitrary 64-bit inputs are prime.
- Use as the primality oracle inside Pollard’s rho factorization.

4.22 Multipoint Evaluation and Interpolation

Recognize. You need to evaluate a degree- n polynomial at m arbitrary points, or reconstruct a polynomial from arbitrary (x_i, y_i) faster than $O(nm)$.

Idea. Build a product tree whose leaves are $(x - x_i)$ and internal nodes are products. For evaluation, propagate polynomial remainders down the tree; each leaf remainder is $P(x_i)$. For interpolation, evaluate the derivative of the total product at x_i , set leaf weights $y_i/M'(x_i)$, then combine upward with sibling product polynomials.

Cost / notes. With fast polynomial multiplication, $O(n \log^2 n)$ -type time and $O(n \log n)$ auxiliary storage (constants depend on implementation). Requires distinct points and field arithmetic for interpolation denominators.

Typical problem statements.

- Evaluate one large polynomial at 10^5 arbitrary coordinates.
- Recover a polynomial from 10^5 value samples.
- Batch many Lagrange queries faster than evaluating them independently.

4.23 NTT (Number Theoretic Transform)

Recognize. You need exact fast polynomial convolution modulo a prime that has a large power-of-two primitive root of unity.

Idea. NTT is FFT performed in a finite field. Use a suitable prime $p = c2^k + 1$ (e.g. 998244353), primitive root powers for butterflies, pointwise multiplication, and an inverse transform multiplying by N^{-1} . For other target moduli, combine convolutions under several NTT primes with CRT.

Cost / notes. $O(N \log N)$ exact modular operations, $O(N)$ memory. Transform length must divide $p - 1$; coefficients must be reduced modulo p .

Typical problem statements.

- Multiply polynomials modulo 998244353.
- Count combinatorial convolutions with no floating error.
- Multiply under an arbitrary modulus using several NTT primes + CRT.

4.24 Polynomial and Formal Power Series Operations

Recognize. A problem asks for inverse, logarithm, exponential, square root, division, derivative/integral, or composition-like operations on long formal power series.

Idea. Treat polynomials modulo x^n and use Newton iteration: for example inverse doubles precision via $g \leftarrow g(2 - fg)$; logarithm uses $\int f'/f$; exponential solves $\log g = f$ by iterative correction. Fast multiplication (NTT/FFT) makes these near-linear. Polynomial division uses reversal + inverse.

Cost / notes. Typically $O(M(n) \log n)$ or $O(M(n))$ for many standard FPS operations, where $M(n)$ is multiplication cost ($O(n \log n)$ with NTT). Preconditions matter: inverse needs $f_0 \neq 0$, log usually $f_0 = 1$, exp usually $f_0 = 0$.

Typical problem statements.

- Compute generating-function coefficients after algebraic transformations.
- Divide/invert large polynomials quickly.
- Solve combinatorial recurrences expressed as $F = \exp(G)$ or similar FPS identities.

4.25 Pollard's Rho Factorization

Recognize. You need to factor a composite 64-bit integer whose prime factors may be far beyond sieve range.

Idea. If Miller–Rabin says n is composite, iterate a pseudorandom polynomial such as $f(x) = x^2 + c \bmod n$ from two states and repeatedly compute $g = \gcd(|x - y|, n)$. A nontrivial gcd reveals a factor because the sequence collides modulo one prime factor earlier than modulo n . Retry constants/seeds when $g = n$. Recurse on factors.

Cost / notes. Heuristic expected about $O(p^{1/2})$ modular steps to find a factor p , often summarized around $O(n^{1/4})$ for balanced semiprimes; extremely effective for 64-bit contest inputs. Needs safe mulmod and primality testing.

Typical problem statements.

- Factor arbitrary 64-bit integers.
- Compute $\varphi(n)$, divisor count, or prime factorization when $n \sim 10^{18}$.

4.26 Product of Two long long Modulo m (Mulmod)

Recognize. You need $(ab) \bmod m$ when a, b, m fit 64 bits but ab may overflow 64-bit multiplication.

Idea. Best in C++ with available `__int128`: cast before multiplying. Portable alternatives use repeated

doubling in $O(\log b)$ or floating/long-double quotient estimation followed by correction. Some 64-bit modular algorithms use Montgomery/Barrett reduction for many products.

Cost / notes. $O(1)$ with 128-bit or quotient-estimation methods; $O(\log m)$ with add-and-double. Correct handling of signed overflow is essential because C++ signed overflow is undefined.

Typical problem statements.

- Modular exponentiation for modulus near 2^{63} .
- Miller–Rabin/Pollard rho on 64-bit numbers.

4.27 Simplex

Recognize. You need to optimize a linear objective under linear inequality/equality constraints over real variables.

Idea. Convert to standard linear-program form and maintain a tableau/basis. Pivot an entering variable that can improve the objective and a leaving variable chosen by the ratio test; Phase I finds a feasible basis if one is not obvious, Phase II optimizes. Detect infeasible and unbounded cases explicitly.

Cost / notes. Polynomial-time algorithms exist, but textbook simplex has exponential worst case; in contest practice it is often fast for moderate dense instances. Floating precision and pivot selection are the main risks.

Typical problem statements.

- Maximize profit with multiple linear resource constraints.
- Solve fractional allocation where variables are continuous rather than integral.

4.28 Chinese Remainder Theorem (CRT)

Recognize. You have congruences $x \equiv a_i \pmod{m_i}$ and want one combined congruence.

Idea. For coprime moduli, merge using modular inverses and the solution is unique modulo $\prod m_i$. Generalized CRT for non-coprime moduli first requires $a_2 - a_1$ divisible by $g = \gcd(m_1, m_2)$, then solves a reduced linear congruence and returns modulus $\text{lcm}(m_1, m_2)$. Merge all equations pairwise.

Cost / notes. $O(k \log M)$ gcd/inverse work for k congruences; big products may overflow. CRT is also used to reconstruct exact/modular convolution results from several NTT primes.

Typical problem statements.

- Find the smallest timestamp satisfying several periodic offset constraints.
- Combine residues modulo different primes.
- Reconstruct an integer coefficient from multiple modular computations.

4.29 Euler's Totient φ

Recognize. $\varphi(n)$, the count of integers in $[1, n]$ coprime to n , or multiplicative-group size/exponent formulas appear.

Idea. From prime factorization $n = \prod p_i^{e_i}$, use $\varphi(n) = n \prod_{p|n} (1 - 1/p)$, implemented as $ans -= ans/p$. For all $1..N$, a sieve can initialize $\text{phi}[i] = i$ and subtract $\text{phi}[j]/p$ on multiples of each prime; linear sieve computes it too.

Cost / notes. $O(\sqrt{n})$ for one number by trial factorization, faster with known/Pollard factors; $O(N \log \log N)$ sieve or $O(N)$ linear sieve for all values. Euler: $a^{\varphi(n)} \equiv 1 \pmod{n}$ when $\gcd(a, n) = 1$.

Typical problem statements.

- Count numbers modulo n that have modular inverses.
- Compute reduced fractions/Farey-related sums using totients.
- Reduce huge exponents modulo $\varphi(m)$ when coprimality conditions hold.

5 Primitives and Reusable Building Blocks

5.1 Modular Arithmetic

Recognize. Many answers are required modulo m , and you want safe reusable addition, subtraction, multiplication, division/inverse, and exponentiation.

Idea. Normalize representatives into $[0, m)$. Implement arithmetic so intermediate values do not overflow; define division as multiplication by a modular inverse only when it exists. A small `modint` type reduces repeated bugs by overloading operators and centralizing normalization/inverse logic.

Cost / notes. Basic operations are $O(1)$ except inverse/power, usually $O(\log m)$. Under prime modulus

every nonzero residue is invertible; under composite modulus invertibility requires gcd 1.

Typical problem statements.

- Any counting DP modulo $10^9 + 7$ or 998244353.
- Polynomial/NTT and combinatorics code using a fixed modulus.
- Modular matrix exponentiation.

5.2 Big Integer

Recognize. Exact integers can exceed 64/128 bits and arbitrary precision is required.

Idea. Represent the magnitude as base- B limbs (e.g. 10^9 or a power of two), least-significant first, plus sign. Implement carry-aware add/subtract, schoolbook or Karatsuba multiplication, comparison and division as needed. In C++, `boost::multiprecision::cpp_int` is usually the fastest contest choice unless custom operations/performance demand otherwise.

Cost / notes. Addition $O(n)$ limbs, schoolbook multiplication $O(n^2)$; library implementations switch algorithms internally. I/O and division can dominate for thousands of digits.

Typical problem statements.

- Compute exact 1000! or huge Fibonacci numbers.
- Compare/evaluate integer formulas exceeding 128-bit range.

5.3 Calendar / Date Arithmetic

Recognize. Problems ask day-of-week, difference between dates, adding days, leap-year handling, or conversion between calendar dates and serial integers.

Idea. Convert $(year, month, day)$ to a monotone “days since epoch” integer using a known civil-date formula, then subtraction/addition is ordinary integer arithmetic; invert the formula to recover a date. Correct Gregorian leap rule: divisible by 4, except centuries unless divisible by 400.

Cost / notes. $O(1)$ per conversion/query. Prefer a proven formula over ad hoc month loops, especially when years may be negative or very large.

Typical problem statements.

- How many days lie between two Gregorian dates?
- What date is K days after a given date?
- Determine weekday from a known epoch.

5.4 Exact Fraction / Rational

Recognize. The answer or intermediate state is an exact rational number and floating-point comparison would be unsafe.

Idea. Store numerator/denominator normalized by gcd, keep denominator positive, and reduce after operations. Compare a/b and c/d by cross multiplication with a widened type or overflow-safe method. For many additions, reducing intermediate gcd factors before multiplying can prevent blow-up.

Cost / notes. Gcd adds $O(\log M)$ arithmetic per normalization; numerator sizes may grow rapidly. Use big integers if 128-bit cross-products are insufficient.

Typical problem statements.

- Compare exact slopes/probabilities without precision error.
- Compute exact geometric intersection coordinates.
- Evaluate rational expressions and output reduced p/q .

5.5 2D Geometry Primitives

Recognize. A geometry problem needs robust points/vectors, orientation, projection, distance, lines, segments, polygons, or convex hull operations.

Idea. Define vector $+/-$, dot and cross products first. Orientation is sign of $\text{cross}(b - a, c - a)$; dot handles projection/angles. Build line/segment intersection, point-on-segment, polygon area, point-in-polygon and convex hull from these few invariants. Keep one consistent EPS policy for floating geometry.

Cost / notes. Most primitives are $O(1)$; convex hull is $O(n \log n)$. The main difficulty is degeneracy: collinearity, endpoint touching, duplicates, parallel/coincident lines and numeric tolerance.

Typical problem statements.

- Test segment intersections and polygon containment.
- Compute convex hull/area/perimeter.

- Find line projections, distances and intersections.

5.6 Integer Geometry Primitives

Recognize. Input coordinates are integral and predicates should be exact.

Idea. Use integer points and perform orientation/cross/dot with a wide type (often `__int128`) so signs are exact. Avoid division until an actual coordinate is needed; compare rational slopes/intersection order by cross multiplication. This eliminates EPS ambiguity for combinatorial geometry.

Cost / notes. $O(1)$ primitives, but choose width from coordinate bounds: a cross product scales like coordinate-difference squared, and further products may need more than 64 bits.

Typical problem statements.

- Exact segment intersection predicates.
- Lattice polygon area via shoelace/Pick’s theorem.
- Convex hull of integer points without floating error.

5.7 3D Geometry Primitives

Recognize. Problems involve points/vectors/planes/lines/spheres in three dimensions.

Idea. Use 3D dot product for angles/projection and vector cross product for normals/areas. Plane $n \cdot x = d$ supports signed distance and line-plane intersection; scalar triple product $a \cdot (b \times c)$ gives oriented volume and coplanarity. Build higher operations from these primitives.

Cost / notes. $O(1)$ per primitive. Floating 3D geometry is numerically fragile; normalize only when necessary and compare squared lengths when possible. Integer predicates may require 128-bit or arbitrary precision.

Typical problem statements.

- Distance/intersection between lines and planes.
- Test coplanarity and compute tetrahedron volume.
- Reflect/project a point onto a plane.

5.8 Matrix Primitive / Matrix Exponentiation

Recognize. A recurrence or state transition is linear and must be applied many times, or generic matrix operations are needed.

Idea. Implement multiplication and identity cleanly; binary exponentiation then computes A^k . A d -state linear recurrence becomes a $d \times d$ transition matrix. Matrix multiplication can also represent path counts, min-plus transitions, Boolean reachability, etc., by changing the underlying semiring.

Cost / notes. Naive multiplication $O(d^3)$ and exponentiation $O(d^3 \log k)$. Sparse/special matrices can be multiplied faster; for one scalar recurrence, linear-recurrence methods may beat matrices.

Typical problem statements.

- Compute the 10^{18} -th term of a small-order recurrence.
- Count walks of exactly k edges between graph vertices.
- Apply a fixed affine/linear transformation repeatedly.

5.9 Matroid / Matroid Intersection

Recognize. You must choose a largest set satisfying two different “independence” systems that behave like linear independence: hereditary sets plus exchange property; common examples are forests and color-capacity constraints.

Idea. A matroid exposes an oracle telling whether adding/swapping an element preserves independence. To intersect two matroids, build an exchange graph around the current common independent set: directed exchanges represent swaps allowed by one matroid or the other. An augmenting path increases the set size by one, analogous to matching.

Cost / notes. The UFMG unweighted intersection implementation is $O(r^2n)$ for ground-set size n and answer rank r with $O(1)$ oracles after builds; weighted variants use shortest paths and cost more. Graphic matroid = acyclic edge sets; partition matroid = color capacities.

Typical problem statements.

- Choose the largest forest whose edges also use each color at most once.
- Select a maximum set simultaneously independent in two partition/graphic constraints.
- Weighted variant: minimum-cost maximum common independent set.

6 Standard Problem Techniques

6.1 Angle Range Intersection

Recognize. Directions/angles are valid only inside circular intervals, and intervals may cross the $0/2\pi$ cut.

Idea. Normalize every angle to one period. A wrapped interval $[l, r]$ with $l > r$ is the union $[l, 2\pi) \cup [0, r]$; after splitting, ordinary interval intersection applies. For many constraints, either keep a small set of non-wrapping arcs and intersect them incrementally, or duplicate angles by $+2\pi$ and choose one consistent unwrapped window.

Cost / notes. $O(1)$ for intersecting two arcs after normalization; $O(k \log k)$ is a convenient bound for k arbitrary arc constraints if endpoints are sorted/merged. Be explicit about closed endpoints and the degenerate “whole circle” case.

Typical problem statements.

- Find headings simultaneously allowed by several angular visibility constraints.
- Decide whether two circular fields of view share a direction.
- Intersect feasible argument ranges of complex/geometric vectors.

6.2 Rectangle Union Area

Recognize. You need the area covered by at least one of many axis-aligned rectangles, with overlaps counted once.

Idea. Sweep in x . Each rectangle creates an add event at its left edge and a remove event at its right edge over a y -interval. Coordinate-compress all y endpoints and maintain with a lazy segment tree the total y -length whose cover count is positive. Between consecutive events, add $\text{covered}Y \cdot \Delta x$ to the answer.

Cost / notes. $O(n \log n)$ time and $O(n)$ memory after compression. In each segment-tree node keep a cover counter; if it is positive the whole node interval is covered, otherwise combine children.

Typical problem statements.

- Compute the painted area of overlapping axis-aligned posters.
- Find the total land covered by rectangular plots.
- Compute union area of many screen rectangles.

6.3 Largest Rectangle in a Histogram

Recognize. Given bar heights, you need the maximum-area rectangle using consecutive bars.

Idea. Maintain a monotone increasing stack of bar indices. When a lower bar arrives, repeatedly pop height h : the current index is the first smaller bar to its right and the new stack top is the first smaller to its left, so the maximal rectangle of height h is known. Append a zero-height sentinel to flush the stack.

Cost / notes. $O(n)$ time and $O(n)$ memory; each index is pushed and popped once. Decide consistently whether equal heights are popped or retained.

Typical problem statements.

- Maximum rectangle of 1s in a binary matrix after treating each row as histogram heights.
- Largest billboard rectangle fitting under a skyline.

6.4 Binomial Coefficient Modulo a Moderate Modulus

Recognize. n and k may be huge, but you need $\binom{n}{k} \bmod m$ where m is small/moderate and may be composite. Ordinary factorials up to n are impossible.

Idea. Factor m into prime powers p^e . For each one, compute the exponent of p in $n!/(k!(n-k)!)$ and the factorial parts with multiples of p removed, using periodic products modulo p^e and recursive quotient blocks. If the exponent is at least e , the residue is zero; otherwise multiply the reduced factorial ratio by p^{exp} . Combine prime-power residues with CRT. For prime m , Lucas’ theorem is the familiar special case.

Cost / notes. Preprocessing/constant factors depend on m ; per query is typically polylogarithmic in n plus work proportional to the prime-power modulus/precomputation. This technique is for huge n , not a replacement for ordinary $O(N)$ factorial tables when N is manageable.

Typical problem statements.

- Compute $\binom{10^{18}}{10^{12}} \bmod 10^5$.
- Answer huge binomial queries modulo a small composite number.

6.5 Closest Pair of Points

Recognize. Among n planar points, find the minimum Euclidean distance between any pair.

Idea. Sort by x and use divide and conquer. Recursively solve left/right halves, let d be the best distance, then merge points by y and inspect only points in the vertical strip within d of the split. In the strip, packing arguments imply each point needs comparison with only a constant number of subsequent y -neighbors.

Cost / notes. $O(n \log n)$ time and $O(n)$ extra memory. Keep points sorted by y through recursion rather than sorting every strip. Squared distances avoid unnecessary square roots.

Typical problem statements.

- Find the nearest pair of cities/particles among up to $2 \cdot 10^5$ coordinates.
- Detect whether any two points are closer than a given threshold.

6.6 Interval Graph Coloring

Recognize. Vertices are intervals; overlapping intervals conflict, and you need the minimum number of colors/resources.

Idea. Sort intervals by starting point. Keep active colors in a min-heap keyed by the end time of the interval currently using each color. Before assigning an interval, recycle any color whose previous interval has ended; otherwise create a new color. Interval graphs are perfect, so this greedy uses exactly the maximum number of simultaneously overlapping intervals.

Cost / notes. $O(n \log n)$ time and $O(n)$ memory. Endpoint convention matters: for closed intervals, $[a, b]$ conflicts with one starting at b ; for half-open intervals it does not.

Typical problem statements.

- Assign the minimum number of rooms to meetings.
- Color time intervals so overlapping jobs use different machines.
- Partition intervals into the minimum number of non-overlapping groups.

6.7 Dynamic Connectivity – Divide and Conquer / Rollback DSU

Recognize. Edges are inserted and deleted over time and connectivity/component queries must be answered offline.

Idea. Turn every edge into the time interval during which it is active. Add that edge to the nodes of a segment tree over time whose ranges are fully covered by its lifetime. DFS the time tree with a rollback DSU: apply all node edges, recurse, answer leaf-time queries, then undo to the saved DSU snapshot. Union by size/rank without path compression makes rollback easy.

Cost / notes. $O((m+q) \log q \log n)$ with a simple rollback DSU analysis, often written near $O((m+q) \log q)$ DSU operations; memory $O(m \log q)$. Requires that the entire operation sequence is known.

Typical problem statements.

- Support add-edge, delete-edge, and “are u, v connected?” queries offline.
- Report the number of connected components after every graph update.

6.8 Dynamic Connectivity – Link-Cut Tree

Recognize. Connectivity must be maintained online under edge insertions/deletions, especially when the maintained graph is a forest or when a more advanced structure maintains a spanning forest.

Idea. For a dynamic forest, represent every tree with a Link-Cut Tree: `link(u,v)` adds an edge between components, `cut(u,v)` removes a tree edge, and `findRoot/access` tests connectivity in logarithmic amortized time. Fully dynamic general-graph variants keep a dynamic spanning forest and additional non-tree edges, finding replacement edges when a tree edge is deleted; the LCT is the tree primitive inside that larger scheme.

Cost / notes. Dynamic-forest operations are $O(\log n)$ amortized. General fully dynamic connectivity has more bookkeeping and implementation-dependent polylogarithmic bounds; do not mistake plain LCT for a complete arbitrary-graph deletion solution.

Typical problem statements.

- Maintain connectivity while edges of a forest are linked and cut online.
- In a dynamic MST/connectivity structure, replace a deleted tree edge by a suitable non-tree edge.

6.9 Maximum-Weight Independent Set in an Interval Graph

Recognize. Each interval has a weight; choose non-overlapping intervals with maximum total weight.

Idea. This is weighted interval scheduling. Sort intervals by right endpoint and for each interval i binary-search $p(i)$, the last interval finishing before i starts. Then $dp[i] = \max(dp[i-1], w_i + dp[p(i)])$. Store choices to reconstruct the selected intervals.

Cost / notes. $O(n \log n)$ time and $O(n)$ memory. The unweighted version reduces to the classic greedy by earliest finishing time; weights require DP.

Typical problem statements.

- Pick non-overlapping paid jobs to maximize revenue.
- Select mutually compatible weighted reservations with maximum total priority.

6.10 Dynamic Convex Hull

Recognize. Points are inserted over time and you repeatedly need hull membership, extreme points, tangents, or support-function queries.

Idea. Maintain upper and lower hull chains ordered by x (or angle). On insertion, locate neighboring points, reject the new point if it lies inside the relevant chain, otherwise insert it and repeatedly erase neighbors that become non-convex according to cross-product signs. Extreme-dot-product queries can binary-search a convex chain when its order is maintained. Fully dynamic deletion needs a more sophisticated structure than this insertion-only recipe.

Cost / notes. Insertion-only ordered-set hulls are typically $O(\log n)$ amortized per insertion plus logarithmic support queries because each obsolete point is erased once. Exact costs depend on the supported operations.

Typical problem statements.

- Insert points and query whether a point lies inside the current convex hull.
- Add lines/points and ask the maximum dot product in a direction.
- Maintain the perimeter/area of an incrementally growing hull.

6.11 Farthest Pair of Points / Convex Diameter

Recognize. Find the maximum Euclidean distance between any two planar points.

Idea. The farthest pair lies on the convex hull. Build the hull, then use rotating calipers: for each hull edge, advance the antipodal pointer while the absolute cross-product area increases. Test the corresponding endpoint pairs and keep the largest squared distance.

Cost / notes. $O(n \log n)$ total for hull construction and $O(h)$ for calipers on h hull vertices. Handle hull sizes 0, 1, 2 separately.

Typical problem statements.

- Find the diameter of a point cloud.
- Choose two stations as far apart as possible among given coordinates.

6.12 Distinct Range Query – Static

Recognize. For many static queries $[l, r]$, return the number of distinct values in the subarray.

Idea. Sort queries by right endpoint. Sweep r from left to right with a Fenwick tree storing a 1 only at the latest occurrence of each value: when value $a[r]$ reappears, erase its previous 1 and add a 1 at r . Then a query ending at r is simply Fenwick sum on $[l, r]$.

Cost / notes. $O((n+q) \log n)$ time, $O(n+q)$ memory after value compression/hash map. Persistent segment trees give an online-by-version alternative.

Typical problem statements.

- Count distinct colors/numbers in many immutable subarray intervals.
- For each query interval, count how many values occur at least once.

6.13 Distinct Range Query with Updates

Recognize. The array changes between queries, yet each query asks the number of distinct values in a range.

Idea. Use Mo’s algorithm with modifications. Record every update with its old/new value and give each query a third coordinate t = number of prior updates. Sort queries in 3D blocks; move L, R with add/remove operations and move time forward/backward by applying/undoing updates, adjusting frequencies when the modified position currently lies inside the range.

Cost / notes. A common block size is about $n^{2/3}$, yielding roughly $O((n+q)n^{2/3})$ pointer/update moves

for balanced workloads. Coordinate-compress all values, including future update values.

Typical problem statements.

- Point assignments mixed with “number of distinct values in $[l, r]$ ” queries.
- Offline dynamic range-frequency statistics where adding/removing one element is $O(1)$.

6.14 Dominator Points (2D Pareto Frontier)

Recognize. Points are ordered by coordinate-wise dominance: (x_1, y_1) dominates (x_2, y_2) when both coordinates are at least as good, and you need the nondominated frontier or dominance queries.

Idea. Sort or maintain frontier points by one coordinate. Along a nondominated staircase, the other coordinate changes monotonically; to insert a point, find its predecessor/successor, discard it if an existing frontier point dominates it, otherwise erase the consecutive frontier points it dominates. This converts many 2D dominance checks into ordered-set predecessor operations.

Cost / notes. Offline frontier construction is $O(n \log n)$; insertion-only maintenance is typically amortized $O(\log n)$ because each deleted frontier point disappears once. Variants can augment gaps/corners with sums or counts.

Typical problem statements.

- Maintain maximal products/solutions under two competing objectives.
- Test whether a new point is dominated by any previously inserted point.
- Compute the Pareto skyline of 2D scores.

6.15 3D Dominance DP

Recognize. A DP state at point $i = (x_i, y_i, z_i)$ depends on the best state among points dominated in all three coordinates.

Idea. Sort by one coordinate and use CDQ divide and conquer on that order. During a merge, process left-half contributors and right-half queries sorted by a second coordinate, while a Fenwick/segment tree over compressed third coordinate stores the best DP value. Roll back/clear updates after each merge. Carefully batch equal coordinates if dominance is non-strict.

Cost / notes. $O(n \log^2 n)$ time and $O(n)$ memory in the standard CDQ+Fenwick implementation. It is the 3D analogue of LIS-style dominance DP.

Typical problem statements.

- Longest chain of boxes where all three dimensions increase.
- $dp[i] = 1 + \max dp[j]$ over all points j with $x_j \leq x_i, y_j \leq y_i, z_j \leq z_i$.

6.16 Gray Code

Recognize. You need to enumerate all B -bit masks so consecutive masks differ in exactly one bit.

Idea. The binary-reflected Gray code is $g(i) = i \oplus (i \gg 1)$ for $0 \leq i < 2^B$. To decode a Gray word g , repeatedly xor it with shifted prefixes (equivalently accumulate prefix xor from the highest bit). The one-bit-change property is ideal for incrementally maintaining a value over all subsets.

Cost / notes. $O(1)$ to map an index to its Gray code and $O(B)$ to decode by the simple loop; enumeration is $O(2^B)$. The sequence is cyclic as well: first and last differ in one bit.

Typical problem statements.

- Enumerate subsets while updating an objective affected by one changed element.
- Construct the required n -bit Gray-code sequence.
- Encode states so adjacent numeric steps flip one bit.

6.17 Half-Plane Intersection

Recognize. You need the polygon satisfying many linear geometric constraints “point lies to the left of directed line L_i ”.

Idea. Sort boundary lines by angle, keeping only the most restrictive among parallel equal-angle lines. Maintain a deque of candidate half-planes; before adding a line, remove lines from each end while their intersection point violates the new half-plane. Finally clean the two ends against each other and intersect consecutive deque lines to obtain the feasible polygon.

Cost / notes. $O(n \log n)$ because of sorting; deque processing is linear. Parallel lines, empty/unbounded intersections, and floating EPS are the difficult cases. A large bounding box can convert an intentionally unbounded region into a bounded polygon when appropriate.

Typical problem statements.

- Intersect many linear visibility/feasibility constraints in the plane.
- Find the kernel of a polygon.
- Compute a convex polygon defined by directed inequalities.

6.18 Heap Sort

Recognize. You need a comparison sort with deterministic $O(n \log n)$ time and $O(1)$ auxiliary memory.

Idea. Build a max-heap in-place using bottom-up sift-down. Repeatedly swap the maximum at index 0 with the last unsorted element, shrink the heap, and sift the new root down. The array suffix grows in sorted order.

Cost / notes. $O(n)$ heap construction, $O(n \log n)$ total, $O(1)$ extra memory. It is not stable and is usually slower in practice than introsort, but its worst-case guarantee is simple.

Typical problem statements.

- Sort an array under strict auxiliary-memory limits.
- Implement the sorting primitive when worst-case $O(n \log n)$ is required without recursion/extra arrays.

6.19 Hungarian Algorithm

Recognize. You need a minimum/maximum-cost one-to-one assignment between two sides of a bipartite graph, usually given as a dense cost matrix.

Idea. Maintain dual potentials for rows/columns and a partial matching. For each left vertex, grow an alternating search using reduced costs, update potentials by the smallest slack until a free right vertex becomes reachable, then augment through stored predecessors. The standard compact implementation solves rectangular matrices too (often requiring $n \leq m$ after transposition).

Cost / notes. $O(n^2 m)$ for an $n \times m$ matrix, $O(n^3)$ when square; $O(n + m)$ extra memory in the common implementation. For sparse cardinality matching use Kuhn/Hopcroft–Karp; for general flow constraints use min-cost flow.

Typical problem statements.

- Assign n workers to n jobs with minimum total cost.
- Choose one column per row and one row per column maximizing total score.

6.20 Inversion Count

Recognize. Count pairs $i < j$ with $a_i > a_j$, or measure how far a permutation is from sorted order.

Idea. During merge sort, when a right-half value is taken before remaining left-half values, it forms inversions with all those remaining values; add their count. Alternatively coordinate-compress values and scan with a Fenwick tree of frequencies.

Cost / notes. $O(n \log n)$ time and $O(n)$ memory with merge sort, or $O(n)$ Fenwick storage after compression. Use 64-bit for the answer since it can reach $n(n-1)/2$.

Typical problem statements.

- Minimum adjacent swaps required to sort a permutation.
- Count out-of-order pairs in an array.
- Compare two rankings by pairwise disagreements.

6.21 LIS – Reconstruct the Sequence

Recognize. You need an actual longest strictly increasing subsequence, not only its length.

Idea. Maintain $tailVal[len]$, the minimum possible ending value of an increasing subsequence of each length, plus $tailIdx[len]$, its array index. For each $a[i]$, binary-search the first tail $\geq a[i]$ (strict LIS), set $parent[i]$ to the index ending the previous length, and replace that tail. Backtrack parents from the final tail index.

Cost / notes. $O(n \log n)$ time and $O(n)$ memory. For nondecreasing LIS, use `upper_bound` instead of `lower_bound`. Replaced tails are not themselves the final subsequence; parent indices are what make reconstruction correct.

Typical problem statements.

- Output one longest increasing subsequence of an array.
- Recover a maximum chain after sorting objects by one coordinate.

6.22 LIS – Length

Recognize. You only need the length of the longest strictly increasing subsequence.

Idea. Maintain an array *tails* where *tails*[*k*] is the smallest ending value seen for an increasing subsequence of length *k* + 1. For each value *x*, replace the first tail $\geq x$; if none exists, append *x*. Smaller tails leave more room for future extensions.

Cost / notes. $O(n \log n)$ time and $O(n)$ memory. Use `lower_bound` for strictly increasing and `upper_bound` for nondecreasing.

Typical problem statements.

- Find the LIS length of up to 10^6 numbers.
- After sorting by one dimension, find the longest chain increasing in another.
- Minimum deletions needed to make a sequence increasing is $n - LIS$.

6.23 Minimum Enclosing Circle

Recognize. Find the smallest circle containing all points.

Idea. Randomly shuffle points and build the answer incrementally. If point p_i lies outside the current circle, the new optimum has p_i on its boundary; restart a nested scan. If another point p_j is outside, the boundary contains p_i, p_j ; if a third p_k is outside that diameter circle, use the circumcircle through the three points. Random order gives the standard expected-linear incremental algorithm.

Cost / notes. Expected $O(n)$ time, $O(1)$ auxiliary state besides the points. Handle collinear triples and use a tolerant inside-circle test; among collinear boundary points the diameter of the farthest pair is enough.

Typical problem statements.

- Smallest-radius transmitter centered anywhere that covers all stations.
- Minimum disk containing a set of planar points.

6.24 Minkowski Sum of Convex Polygons

Recognize. You need $A + B = \{a + b : a \in A, b \in B\}$ for convex polygons, often for collision/configuration-space geometry.

Idea. Put both convex polygons in counterclockwise order and rotate each list so its lexicographically lowest vertex comes first. Their edge vectors are angle-sorted around the polygon; merge the two edge-vector sequences by cross-product sign, adding parallel vectors together, while accumulating vertices from the sum of the starting points.

Cost / notes. $O(n + m)$ after canonicalizing already-convex polygons; $O((n + m) \log(n + m))$ if hulls must first be built. Remove redundant collinear points consistently.

Typical problem statements.

- Test whether two convex shapes can overlap after a translation by checking a Minkowski difference.
- Add all pairwise vectors from two convex polygons without enumerating nm pairs.

6.25 Mo's Algorithm

Recognize. There are many offline range queries on a static array, and the answer can be updated cheaply when one endpoint moves by one position.

Idea. Choose block size about $\sqrt{n}$. Sort queries by block of *L*, then by *R* (alternating *R* direction between blocks often improves constants). Maintain a current interval and answer; move *L, R* with `add/remove` functions until it equals each query. Mo trades an easy dynamic update rule for a clever query order.

Cost / notes. About $O((n + q)\sqrt{n} \cdot F)$ where *F* is add/remove cost; often $O((n + q)\sqrt{n})$ with $O(1)$ updates. It is offline and usually loses to a logarithmic data structure if one exists.

Typical problem statements.

- Answer range mode/frequency-power queries with easy add/remove but hard merges.
- Count equal pairs or distinct values in many static subarrays.

6.26 Mo's Algorithm + Rollback DSU

Recognize. Each query concerns a contiguous interval of graph edges, and the answer is connectivity information that DSU can add efficiently but cannot delete normally.

Idea. Sort interval queries in Mo-style blocks. Within a left-end block, keep a DSU containing the long right-side portion and only extend *R* permanently; for the short left-side prefix of each query, take a rollback snapshot, add those edges temporarily, read the component/connectivity answer, then rollback. Rebuild the

permanent DSU when moving to the next block.

Cost / notes. The UFMG-style component-count implementation is about $O(m\sqrt{q} \log n)$; exact bounds vary with block choice and rollback-DSU cost. The key trick is to arrange movement so expensive deletions become rollback to a snapshot.

Typical problem statements.

- For every query [*l, r*] over an edge list, report the number of connected components using only those edges.
- Answer interval-edge connectivity queries offline without a fully dynamic graph structure.

6.27 Mo's Algorithm on Trees

Recognize. Path/subtree queries on a static tree have an answer that can be maintained by toggling one vertex at a time.

Idea. Flatten the tree with an Euler tour where each vertex appears on entry and exit. A path (*u, v*) can be converted to one interval in this doubled tour after ordering by tin; vertices appearing twice cancel under a toggle, while vertices appearing once are exactly the path, except possibly the LCA which is added separately. Sort these intervals with Mo and toggle vertices as endpoints move. Subtree queries are even simpler with a one-entry Euler order.

Cost / notes. Typically $O((n + q)\sqrt{n} \cdot F)$ toggle work. Precompute LCA in $O(n \log n)$ or better. Correct interval conversion is the main source of bugs.

Typical problem statements.

- Count distinct colors on many tree paths.
- Compute a frequency statistic on arbitrary *u–v* paths offline.
- Answer subtree statistics with no updates using Euler-tour Mo.

6.28 Palindromic Factorization

Recognize. Partition a string into the minimum number of palindromic substrings (or count/optimize palindrome partitions).

Idea. A direct DP is $dp[i] = 1 + \min dp[j]$ over palindromic suffixes $s[j..i]$. The challenge is enumerating palindromic suffixes efficiently: an eertree exposes them through suffix links, and series links/grouped equal-difference links let the minimum-partition DP reuse values rather than walking all palindromic suffixes every time. For moderate *n*, Manacher plus explicit transitions may be simpler.

Cost / notes. Naive palindrome DP is $O(n^2)$. Eertree-based optimized factorization can be near-linear/ $O(n \log n)$ depending on the series-link implementation. Use the notebook code when constraints demand the optimized variant.

Typical problem statements.

- Minimum number of palindromic pieces whose concatenation is the whole string.
- Decide whether a string can be partitioned into at most *k* palindromes.

6.29 Expression Parsing

Recognize. Input is an arithmetic/Boolean expression with parentheses, precedence, associativity, and possibly unary operators/functions.

Idea. Either use recursive descent with one function per precedence level, or the shunting-yard algorithm: operators sit on a stack and are popped while they bind more tightly than the incoming operator; operands go to an output/RPN stack. Evaluate while parsing or build an AST. Tokenize first so multi-digit numbers and unary minus are unambiguous.

Cost / notes. $O(n)$ time and $O(n)$ stack/AST space. Explicitly encode right-associative operators (e.g. exponentiation) and distinguish unary from binary operators by parser context.

Typical problem statements.

- Evaluate a fully general arithmetic expression string.
- Convert infix notation to postfix/RPN.
- Parse formulas to build an expression tree for later simplification.

6.30 Offline RMQ with Divide and Conquer

Recognize. All range-minimum queries are known in advance, and you want a simple offline solution without a persistent/tree query structure.

Idea. Recursively split the array at midpoint *M*. Precompute suffix minima from *M* to the left and prefix

minima from $M + 1$ to the right; every query crossing the split is answered immediately by combining those two values. Partition non-crossing queries into the left/right recursive calls.

Cost / notes. $O(n \log n + q \log n)$ with straightforward per-level preprocessing/partitioning; the notebook-style formulation is commonly summarized as $O((n + q) \log n)$, with linear work per recursion level. Memory can be linear with careful buffers.

Typical problem statements.

- Answer a batch of immutable minimum/maximum range queries offline.
- Use the same divide-and-conquer pattern for an associative range statistic whose crossing answer is easy.

6.31 Segment Intersection

Recognize. Given two line segments, determine whether they intersect and possibly compute the intersection point.

Idea. For exact integer predicates, compute orientations $orient(a, b, c)$ via cross products. In the proper case, segments intersect when endpoints of each segment lie on opposite sides of the other line. Separately handle zero-orientation collinear cases with bounding-box/on-segment tests. For a coordinate, solve the two parametric lines after the predicate says an intersection exists.

Cost / notes. $O(1)$ per pair. Use wide integer arithmetic for cross products; with floating coordinates use a consistent EPS. Distinguish proper crossing, endpoint touching, and overlapping collinear segments if the problem cares.

Typical problem statements.

- Test whether two road segments cross.
- Detect polygon self-intersections.
- Compute the point where two nonparallel segments meet.

6.32 de Bruijn Sequence

Recognize. Construct a shortest cyclic string over an alphabet of size k containing every length- n word exactly once as a cyclic substring.

Idea. Build the de Bruijn directed graph: vertices are length- $(n - 1)$ strings and each length- n string is an edge from its prefix to its suffix. Every vertex has equal in/out degree and the graph is connected, so an Euler cycle lists every edge exactly once. Hierholzer’s algorithm yields the sequence; append the starting $(n - 1)$ symbols if a linear representation is required.

Cost / notes. $O(k^n)$ time and output size, since there are exactly k^n required length- n words. The graph can be represented by base- k integers instead of explicit strings.

Typical problem statements.

- Construct a binary string containing every n -bit pattern exactly once cyclically.
- Generate a minimal “combination lock” sequence that visits every length- n code.

6.33 Shortest Addition Chain

Recognize. To compute x^n with the fewest multiplications, find the shortest sequence $1 = a_0 < a_1 < \dots < a_t = n$ where every new term is the sum of two earlier terms.

Idea. Use iterative deepening DFS on the chain length. At depth d , try promising sums of existing chain elements, largest first, while deduplicating equal sums. A strong prune is: if even doubling the current maximum at every remaining step cannot reach n , abandon the branch. The first successful depth is optimal.

Cost / notes. Exponential/backtracking in the worst case, but strong pruning solves the small/moderate targets for which this technique is used. There is no simple greedy guaranteeing shortest chains.

Typical problem statements.

- Minimize multiplications needed to exponentiate to a fixed integer exponent.
- Find the minimum number of “add two previously built values” operations needed to construct n .

6.34 Simple Polygon Verification

Recognize. Given polygon vertices in boundary order, determine whether the polygon is simple: nonadjacent edges must not intersect.

Idea. Create all polygon edges and run a sweep-line segment-intersection test. Events are segment endpoints; the active set is ordered by segment height at the sweep x . New/deleted segments only need to be checked against neighboring active segments. Ignore the legal shared endpoint of adjacent polygon edges, but reject other crossings/touches according to the problem definition.

Cost / notes. $O(n \log n)$ with a robust sweep; $O(n^2)$ pairwise checking is often preferable for small n . Degenerate vertical/collinear segments make sweep implementations subtle.

Typical problem statements.

- Validate that a proposed polygon boundary does not cross itself.
- Decide whether a sequence of fence vertices forms a simple polygon.

6.35 Steiner Tree – Small Number of Terminals

Recognize. In a weighted graph, connect a small required set of k terminals with minimum total edge cost; extra nonterminal vertices may be used.

Idea. Let $dp[mask][v]$ be the minimum cost of a connected subgraph joining terminals in $mask$ and ending/meeting at v . Combine two submasks at the same v : $dp[mask][v] = \min(dp[sub][v] + dp[mask \setminus sub][v])$, then propagate each mask through graph shortest paths (Dijkstra for nonnegative weights). Singleton masks start from their terminal.

Cost / notes. Typical bound $O(3^k n + 2^k(m \log n))$, memory $O(2^k n)$. Feasible only when the number of terminals is small, even if the graph itself is large.

Typical problem statements.

- Connect up to 15 required cities with minimum network cost, allowing intermediate cities.
- Minimum-cost pipe/cable tree connecting a small set of mandatory vertices.

6.36 Sweep Direction

Recognize. As a projection direction rotates, you need every ordering of a fixed point set induced by projection onto that direction, or need to process events exactly when two points swap order.

Idea. For each pair of points, compute the direction at which their projections become equal; these $O(n^2)$ directions are swap events. Sort events by angle and start with the point order for one direction. Sweeping through event angles, swap the affected pair(s) in the maintained permutation; between events the projection order is unchanged. Batch simultaneous/parallel events when degeneracies matter.

Cost / notes. $O(n^2 \log n)$ for sorting pair events and near $O(1)$ per swap with position arrays. This is a geometric analogue of sweeping all permutation changes rather than recomputing a sort for every direction.

Typical problem statements.

- Optimize a score that depends on the order of points along every possible projection direction.
- Enumerate all distinct point orderings produced by rotating a line.

6.37 Delaunay Triangulation

Recognize. You need a triangulation in which no point lies inside the circumcircle of any triangle; this often supports nearest-neighbor/Voronoi geometry.

Idea. The Delaunay triangulation is dual to the Voronoi diagram. A common contest implementation is divide-and-conquer or randomized incremental insertion with edge flips: after inserting a point, any edge violating the local empty-circumcircle test is flipped until all edges are locally Delaunay. Robust orientation and in-circle predicates are central.

Cost / notes. $O(n \log n)$ expected/standard time and $O(n)$ edges for nondegenerate planar input. Exact predicates are strongly preferable when coordinates are integral; cocircular points make the triangulation non-unique.

Typical problem statements.

- Build the planar graph containing every Euclidean nearest-neighbor candidate.
- Derive Voronoi adjacency or closest-pair structures.
- Compute geometric constructions requiring a sparse triangulation of points.

6.38 Triangle Counting in Graphs

Recognize. Count triples of vertices whose three pairwise edges exist.

Idea. Orient every undirected edge from the lower (*degree, vertexId*) endpoint to the higher one. The resulting forward out-degree is $O(\sqrt{m})$ in the standard analysis. For every directed edge $u \rightarrow v$, mark/iterate forward neighbors and count common forward neighbors; each triangle is counted exactly once at its smallest oriented vertex.

Cost / notes. $O(m\sqrt{m})$ worst-case with degree orientation and $O(n + m)$ memory; simpler sorted-neighbor intersection gives $O(\sum_{uv \in E} \min(deg_u, deg_v))$.

Typical problem statements.

- Count all triangles in a sparse social network.
- Determine how many 3-cliques exist in an undirected graph.

7 Strings

7.1 Aho–Corasick

Recognize. You must search a text for many patterns simultaneously.

Idea. Build a trie of all patterns. BFS computes each node’s failure link to the longest proper suffix that is also a trie prefix; missing transitions can inherit transitions through failure links. While scanning text, follow one automaton transition per character. Pattern matches at a state include that state and its failure/output ancestors; aggregate counts along the failure tree if you only need occurrence totals.

Cost / notes. $O(\text{total pattern length} + |text| + \#matches)$ with explicit alphabet transitions; memory is the trie size times transition representation. Failure-link count propagation can avoid enumerating every match online.

Typical problem statements.

- Count occurrences of thousands of dictionary words in one text.
- Find every forbidden pattern while scanning a string/DP automaton state.
- Compute how many patterns end at each text position.

7.2 Eertree / Palindromic Tree

Recognize. You need all distinct palindromic substrings and palindrome suffix information while scanning a string.

Idea. Each node represents one distinct palindrome and stores its length plus a suffix link to its longest proper palindromic suffix. Keep two roots of lengths -1 and 0 . When appending character c , follow suffix links until the current palindrome can be extended by c on both sides; reuse or create that child, then compute its suffix link similarly.

Cost / notes. At most $n + 2$ nodes. Construction is $O(n \cdot \text{lookup})$, effectively $O(n)$ with array transitions on a fixed alphabet. Occurrence counts, number of distinct palindromes, longest palindromic suffix and palindrome DP can be attached to nodes.

Typical problem statements.

- Count distinct palindromic substrings.
- For every prefix, find/count palindromic suffixes.
- Optimize a DP over palindrome partitions.

7.3 KMP / Prefix Function

Recognize. Find occurrences of one pattern in a text, or reason about borders/periods of a string.

Idea. For string s , prefix function $\pi[i]$ is the length of the longest proper prefix equal to a suffix ending at i . When adding $s[i]$, repeatedly fall back $j = \pi[j - 1]$ until characters match, then increment. Search by running the same automaton on pattern + separator + text, or maintain pattern state directly.

Cost / notes. $O(n + m)$ time and $O(m)$ memory for pattern/text matching. Each fallback across the whole run is amortized linear. Borders are obtained by repeatedly following $\pi[len - 1]$.

Typical problem statements.

- Find all occurrences of pattern P in text T .
- Find all border lengths / the smallest period of a string.
- Build a DFA-like matcher for DP over forbidden prefixes.

7.4 Manacher’s Algorithm

Recognize. You need the palindrome radius around every center, or the longest palindromic substring, in linear time.

Idea. Maintain the rightmost palindrome interval. For a new center inside it, initialize its radius from the mirrored center, capped by the current right boundary, then expand explicitly. If expansion passes the boundary, update the rightmost interval. Usually maintain separate arrays for odd and even centers.

Cost / notes. $O(n)$ time and $O(n)$ memory. From radii you can test whether any substring is a palindrome in $O(1)$ after careful index conversion.

Typical problem statements.

- Find the longest palindromic substring.
- Count all palindromic substrings from their center radii.
- Answer many “is $s[l..r]$ a palindrome?” queries.

7.5 Minimum / Maximum Suffix and Cyclic Shift

Recognize. You need the lexicographically smallest/largest suffix or rotation without building a full suffix array.

Idea. For minimum cyclic rotation, Booth’s algorithm compares two candidate starts in $s + s$: after a mismatch k characters later, the lexicographically worse candidate and all starts before the mismatch can be skipped. Duval’s Lyndon-factorization viewpoint gives an equally linear solution. Minimum suffix is a closely related two-candidate/Lyndon scan; reverse comparisons for maximum.

Cost / notes. $O(n)$ time and $O(1)$ or $O(n)$ auxiliary memory depending on whether $s + s$ is materialized. Be precise about rotations versus suffixes: rotations wrap around, suffixes do not.

Typical problem statements.

- Output the lexicographically smallest rotation of a circular string.
- Find the smallest suffix/start position in linear time.
- Canonicalize a cyclic necklace for comparison.

7.6 String Hashing

Recognize. You need fast substring equality/LCP/palindrome checks and can tolerate a tiny collision probability.

Idea. Polynomial rolling hash treats characters as coefficients: precompute powers p^i and prefix hashes. A substring hash is a prefix difference, optionally normalized to a common exponent. For concatenation, multiply one component by an appropriate power before adding the other. Randomize the base and/or use two independent moduli to make adversarial collisions unlikely.

Cost / notes. $O(n)$ preprocessing and $O(1)$ substring hash; equality is probabilistic unless you verify characters. Two 32-bit/64-bit independent hashes are common when correctness risk matters.

Typical problem statements.

- Compare arbitrary substrings in $O(1)$.
- Binary-search longest common prefixes of suffixes.
- Detect repeated substrings or palindromes using forward/reverse hashes.

7.7 String Hashing Modulo $2^{61} - 1$

Recognize. You want a fast single 64-bit polynomial hash with a very large prime modulus and efficient multiplication reduction.

Idea. Use the Mersenne prime $M = 2^{61} - 1$. Multiply with 128-bit intermediate, then exploit $2^{61} \equiv 1 \pmod{M}$: fold high bits into low bits and subtract M if needed. Build prefix/power hashes exactly like ordinary polynomial hashing. A randomized base greatly reduces structured/adversarial collisions.

Cost / notes. $O(n)$ preprocessing, $O(1)$ substring hash, and very small collision probability (still not deterministic equality). The specialized reduction is faster than generic modulo on many platforms.

Typical problem statements.

- High-performance substring equality for long strings.
- Hash tree/string states where two-modulus hashing is too expensive but one large prime is acceptable.

7.8 Suffix Array – $O(n \log n)$ Doubling

Recognize. You need all suffixes sorted lexicographically and want a simple reliable construction.

Idea. Initially rank suffixes by the first character. At doubling step k , rank suffix i by pair $(rank[i], rank[i + 2^k])$; sort these pairs, compress equal pairs to new integer ranks, and double the compared prefix length. Counting/radix sort on integer ranks keeps each round linear. Build LCP afterward with Kasai in $O(n)$.

Cost / notes. $O(n \log n)$ with radix/counting sort (or $O(n \log^2 n)$ with comparison sort), $O(n)$ memory. Append/use a unique sentinel or carefully define out-of-range rank.

Typical problem statements.

- Sort all suffixes and answer substring-occurrence queries by binary search.
- Find the longest repeated substring via maximum adjacent LCP.
- Count distinct substrings using suffix lengths minus LCPs.

7.9 Suffix Array – Linear Time (SA-IS / Induced Sorting)

Recognize. Input is large enough that $O(n \log n)$ suffix-array construction is undesirable, and a tested linear implementation is available.

Idea. SA-IS classifies suffix positions as S-type or L-type and singles out LMS positions. Sort LMS substrings (recursively compressing them to a smaller string if their names are not unique), place them into character buckets, then *induce* the order of L- and S-suffixes from already placed suffixes. The recursion shrinks to the LMS subset.

Cost / notes. $O(n + \sigma)$ time and $O(n + \sigma)$ memory with integer alphabet in $[0, \sigma)$. The implementation is intricate; during ICPC, treat a proven template as a black box and use doubling unless constraints actually require linear time.

Typical problem statements.

- Construct a suffix array for multi-million-character input.
- Use a library suffix-array primitive where asymptotic linear preprocessing is required.

7.10 Dynamic Suffix Array

Recognize. A string changes under a restricted update model and you need to maintain/search the lexicographic order of its suffixes without rebuilding from scratch.

Idea. Dynamic suffix-array implementations are specialized: maintain an order label/rank for each suffix in an ordered structure, and compare newly affected suffixes through their first character plus the rank/hash/order of the remaining suffix. Updates then relabel or insert affected suffixes, often using balanced trees/order-maintenance and hashing/LCP support. The exact supported operation (commonly adding characters at one end rather than arbitrary edits) is part of the template contract.

Cost / notes. Complexity is implementation-specific, usually polylogarithmic per supported update/query after preprocessing. This is not a generic “edit any character and magically maintain SA” tool; read the code interface before using it.

Typical problem statements.

- Repeatedly prepend/append characters and query the rank/order of current suffixes.
- Maintain suffix-order-based queries on an incrementally constructed string.

7.11 Suffix Automaton (SAM)

Recognize. You need a compact automaton representing every substring of one string, especially for distinct-substring, occurrence, or longest-common-substring tasks.

Idea. A state represents an equivalence class of substrings sharing the same set of ending positions; store *len*, suffix *link*, and transitions. Appending a character creates a new terminal state; if the required suffix transition would violate the length invariant, clone an existing state with shorter *len* and redirect transitions/links. Every substring corresponds to a path from the initial state.

Cost / notes. At most $2n - 1$ states, $O(n \cdot \text{lookup})$ construction (linear with fixed-alphabet arrays). State v contributes $\text{len}[v] - \text{len}[\text{link}[v]]$ distinct substrings; propagate terminal occurrence counts in decreasing *len* order.

Typical problem statements.

- Count the number of distinct substrings.
- Find the longest common substring of two strings.
- Count occurrences of every substring represented by a SAM state.

7.12 Trie

Recognize. You store many strings and need prefix-based insertion/search/counting.

Idea. Each node represents a prefix; outgoing edges are characters. Insert by following/creating edges and store terminal counts/IDs, subtree counts, or other aggregates. Prefix query is just walking the characters; lexicographic traversal follows outgoing edges in character order. Tries are also the base structure behind Aho–Corasick and binary xor tries.

Cost / notes. $O(|s|)$ per insertion/query, with memory proportional to the total number of distinct stored prefixes. Array transitions are fast for small alphabets; maps/vectors save memory for sparse large alphabets.

Typical problem statements.

- Count how many dictionary words share a queried prefix.

- Find whether a word is present in a dictionary.
- Binary trie: maximize $x \oplus y$ over inserted integers.

7.13 Z Algorithm

Recognize. For every position i , you need the longest prefix of the string matching the substring starting at i .

Idea. Maintain the rightmost matching segment $[L, R)$. If $i < R$, initialize $z[i]$ with $\min(R - i, z[i - L])$ from the mirrored prefix position, then extend by direct comparisons; if the match passes R , update L, R . Pattern matching uses $P + \# + T$: positions whose Z value reaches $|P|$ are occurrences.

Cost / notes. $O(n)$ time and $O(n)$ memory because every explicit extension increases the global right boundary. Z and prefix-function encode closely related border information; use whichever fits the recurrence more naturally.

Typical problem statements.

- Find all occurrences of a pattern in a text.
- For every suffix, compute its LCP with the whole string.
- Detect borders/periodicity and repetitions.

Extra Utilities

debug.cpp

A contest-only debugging helper: compact printers/macros for scalars, pairs, containers and named expressions. Keep it compile-time removable (for example behind `#ifdef LOCAL`) so submissions have no debug output or performance cost. Typical uses are inspecting a DP row/graph adjacency during local testing and printing several named variables at a failing state.

fastIO.cpp

A custom buffered scanner/writer for inputs so large that ordinary synchronized iostreams may be a bottleneck. In most C++ ICPC problems, `ios::sync_with_stdio(false); cin.tie(nullptr);` is already sufficient; use a manual byte buffer only when profiling/constraints justify it. Typical uses are parsing millions of integers or producing very large numeric output.

Notebook naming note. Ambiguous/specialized names were interpreted according to the public UFMG Biblioteca index and the intended competitive-programming use of those templates. Always check a template’s exact interface and invariants before adapting it under contest pressure.